Software Design Document

WeCureIT

Developed by:
Sumedha Reddy Mannem

Distribution clause
This document was created as a deliverable for the Fall 2024 CSCi 6235 course at the George Washington University, 1918 F Street, NW, Washington, DC 20052. Distribution is unrestricted provided the source is referenced.

REVISION HISTORY

Revision	Date	Affected Pages	Explanation of Change
V1	11/15/2024		Initial draft

TABLE OF CONTENTS

Scope…….……4
Overview……4
Theoretical background and definition of terms……4
Software Architecture……5
Software Bill Of Materials (SBOM)……..6
File Structure……..……7
Class structure……8
Data Design……9
Object structure……..9
Database Model……13
Inter-Process Communication……...14
Software Functionality……..15
Use Cases……..15
Function execution……18

Scope

The WeCureIT medical clinic reservation system is a comprehensive platform designed to facilitate appointment scheduling by centralizing healthcare services into a single, user-friendly interface. This system is accessible to three main user groups: Patients, Doctors, and Administrators, with each group having a tailored view and functionality to meet their specific needs. Patients can log in, select their required speciality, and view facilities offering relevant services, while Doctors can set their preferences for facility assignments and specify their working hours. Administrators have the exclusive ability to create, update, and manage facilities and doctor profiles within the system, supporting the smooth functioning of healthcare services. Additionally, patients benefit from flexible scheduling options, allowing them to choose appointment date, time, and duration, with cancellation policies that enable fee-free cancellations up to 24 hours in advance. While the WeCureIT system prioritizes accessibility and usability across user groups, it does not include Health Insurance Portability and Accountability Act (HIPAA) compliance, as this component was intentionally excluded during design.

Overview

The WeCureIT medical clinic reservation system is to streamline the process of healthcare appointment scheduling by offering a dynamic, patient-centric platform. This system enables doctors to set preferences for the facilities they wish to work in and define their daily working hours, allowing efficient and doctor allocation across multiple locations. Patients can log in, select a required medical speciality, and view facilities that offer relevant services. The system then provides available doctors based on the chosen speciality and facility, enabling patients to make informed choices. Patients can further customize their appointments by selecting their preferred date, time, and appointment slot duration (15, 30, or 60 minutes). The system also includes cancellation policies, allowing patients to cancel appointments without fees if done 24 hours in advance, while an automated notification system alerts patients in case of doctor cancellations. Through this design, the WeCureIT system ensures optimized resource allocation, improved scheduling flexibility, and enhanced patient-doctor interaction.

Theoretical background and definition of terms

For the WeCureIT project, the website is developed using the MERN stack (MongoDB, Express, React, and Node.js), ensuring a seamless, scalable, and dynamic application. MongoDB provides a flexible database structure to store user information, doctor schedules, facility details, and appointments. Express and Node.js handle the server-side operations and API interactions, while React powers the frontend, delivering an interactive and responsive user interface. The users in this website includes:

- **Patients:** Users who wish to schedule appointments with WeCureIT doctors based on speciality and facility.
- **Doctors:** Medical professionals with specific specialties available to treat patients at designated WeCureIT facilities.
- **Administrators:** Individuals responsible for maintaining up-to-date information on doctors, specialties, and facility availability, ensuring the platform functions smoothly and reliably.

Each user group is provided with tailored functionalities to meet WeCureIT's goals of accessible and efficient healthcare scheduling.

Software Architecture

The WeCureIT project architecture utilizes a decoupled architecture with distinct components to manage different layers of the application, enhancing maintainability, scalability, and development efficiency. The architecture consists of:

- **Frontend (React.js):** This layer is responsible for creating a dynamic and responsive user interface, allowing users (Patients, Doctors, and Administrators) to interact with the system. It includes components for routing, UI elements, and service functions, with Axios used to send HTTP requests (GET, POST, PUT, DELETE) to the backend.
- **Backend (Node.js with Express):** Node.js and Express handle the business logic and data processing, managing REST API endpoints to interact with the frontend. The backend uses controller logic and middleware to process incoming requests and ensure secure, reliable operations.
- **Database (MongoDB):** MongoDB stores the application's data, including patients' appointment schedules, doctor availability, facility details, and other critical information. The backend communicates with MongoDB through database queries to retrieve and update data as needed.

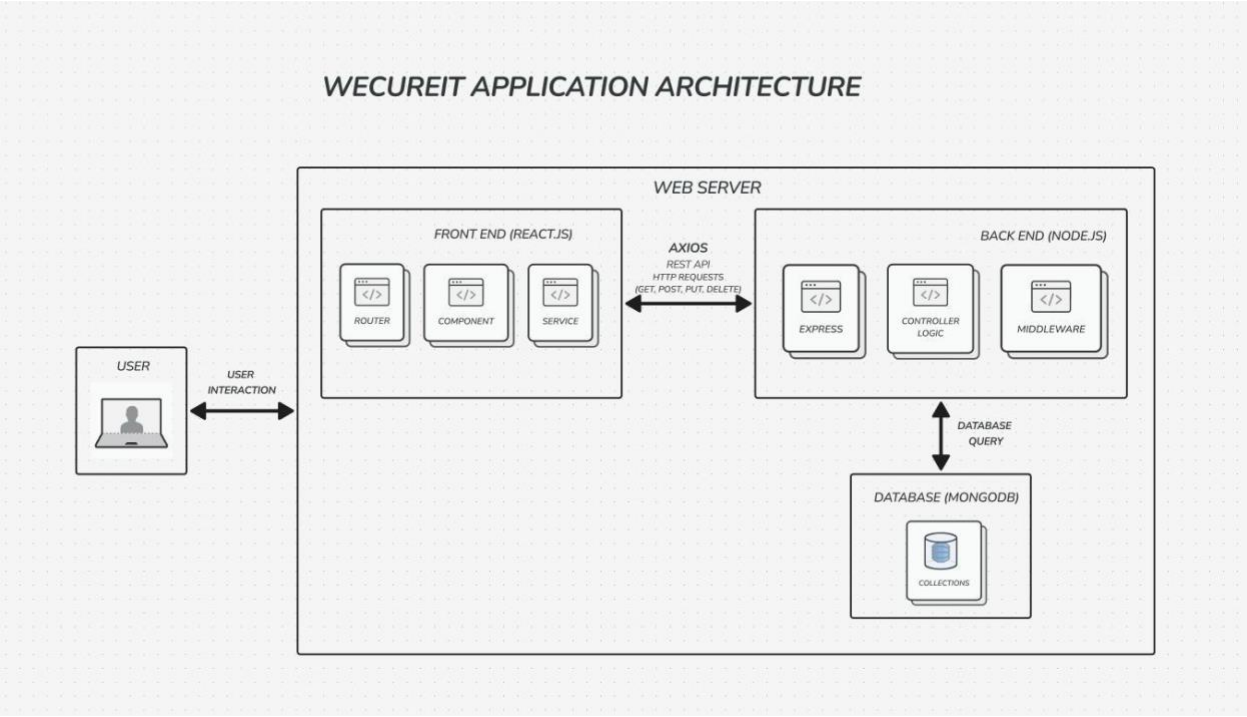

Figure 1: *Entire inter-relationship between different components*

Software Bill of Materials (SBOM)

The following table lists all the software components used to develop the WeCureIT medical clinic reservation system. All these components are available free of charge, either as open-source software or free-to-use services.

Component	Version	Usage	Source URL
MongoDB	4.x or 5.x	Database to store and manage data such as patient details, appointments, and doctor information.	https://www.mongodb.com/
Express	4.x	Backend framework for handling HTTP requests and building RESTful APIs.	https://expressjs.com/
React	17.x or 18.x	Frontend framework for building a dynamic and responsive user interface.	https://reactjs.org/
Node.js	16.x or 18.x	JavaScript runtime for backend server, handling asynchronous operations and API requests.	https://nodejs.org/
Axios	0.27.x	Library for making HTTP requests from the frontend to the backend APIs.	https://axios-http.com/
Mongoose	6.x	ODM library for MongoDB, used to manage and interact with MongoDB collections in a more structured way.	https://mongoosejs.com/
Tailwind CSS	3.x	CSS framework to style the frontend with a utility-first approach.	https://tailwindcss.com/
Google Fonts	N/A	For sending email notifications (such as appointment confirmations or cancellations).	https://fonts.google.com/
Dotenv	16.x	To load environment variables from a .env file, keeping sensitive data secure.	https://github.com/motdotla/dotenv

File Structure

The project is divided into two parts: frontend and backend. Each part has its own dedicated folder structure, organized as follows:

```
└── Frontend/
    ├── build
    ├── node_modules
    ├── src/
    │   ├── doctorMode/
    │   ├── adminMode/
    │   ├── patientMode/
    │   ├── components/
    │   ├── styles/
    │   ├── App.jsx
    │   └── index.jsx
    ├── package-lock.json
    ├── package.json
    └── tailwind.config.js


└── Backend/
    ├── models
    ├── node_modules
    ├── routes
    │   ├── doctorModeRoutes/
    │   ├── adminModeRoutes/
    │   ├── patientModeRoutes/
    ├── .env
    ├── app.js
    ├── package-lock.json
    └── package.json
```

Class Structure

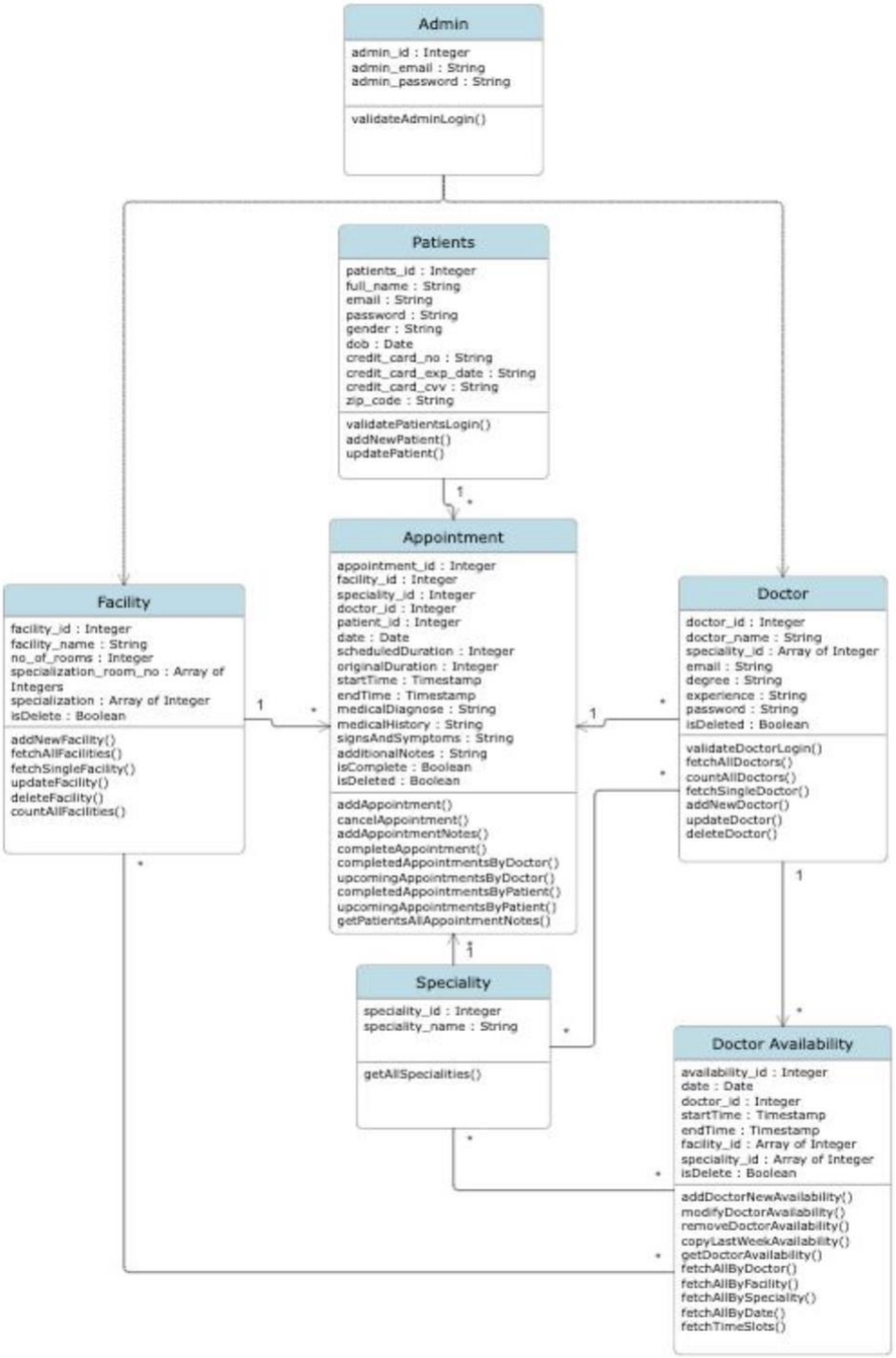

Figure 2: *Class Diagram*

Data Design

Object structure

1. Admin Object

Attributes:
- admin_id: Unique identifier for the admin.
- admin_email: Email address of the admin.
- admin_password: Password for admin access.

Methods:
- validateAdminLogin(): Method to validate the admin's login credentials.

2. Patients Object

Attributes:
- patient_id: Unique identifier for the patient.
- full_name: Full name of the patient.
- email: Email address of the patient.
- password: Password for patient access.
- gender: Gender of the patient.
- dob: Date of birth.
- credit_card_no: Credit card number.
- credit_card_exp_date: Expiration date of the credit card.
- credit_card_cvv: CVV code of the credit card.
- zip_code: ZIP code of the patient.

Methods:
- validatePatientLogin(): Method to validate the patient's login credentials.
- addNewPatient(): Method to add a new patient to the database.
- updatePatient(): Method to update patient details.

3. Facility Object

Attributes:
- facility_id: Unique identifier for the facility.
- facility_name: Name of the facility.
- no_of_rooms: Total number of rooms in the facility.
- specialization: Array of integers representing specialties available in the facility.
- is_delete: Boolean indicating if the facility is deleted.

Methods:
- addNewFacility(): Method to add a new facility.
- fetchAllFacilities(): Method to retrieve all facilities.
- getFacility(): Method to get a specific facility.

- updateFacility(): Method to update facility details.
- deleteFacility(): Method to delete a facility.
- countAllFacilities(): Method to count the total facilities.

4. Appointment Object

Attributes:

- appointment_id: Unique identifier for the appointment.
- facility_id: ID of the facility where the appointment is booked.
- speciality_id: ID representing the speciality required.
- doctor_id: ID of the doctor for the appointment.
- patient_id: ID of the patient who booked the appointment.
- date: Date of the appointment.
- slot_duration: Duration of the appointment slot.
- start_time: Start time of the appointment.
- end_time: End time of the appointment.
- is_delete: Boolean indicating if the appointment is deleted.
- status: Status of the appointment (e.g., confirmed, pending).

Methods:

- addAppointment(): Method to add a new appointment.
- cancelAppointment(): Method to cancel an appointment.
- completeAppointment(): Method to mark an appointment as completed.
- fetchAppointmentsByDoctor(): Method to get appointments by a doctor.
- fetchAppointmentsByPatient(): Method to get appointments by a patient.
- getPendingAppointments(): Method to get pending appointments.
- getCompletedAppointments(): Method to get completed appointments.

5. Doctor Object

Attributes:

- doctor_id: Unique identifier for the doctor.
- doctor_name: Name of the doctor.
- speciality_id: Speciality ID for the doctor.
- experience: Years of experience.
- rating: Rating of the doctor.
- gender: Gender of the doctor.
- phone: Contact number.

Methods:

- validateDoctorLogin(): Method to validate the doctor’s login credentials.
- fetchAllDoctors(): Method to retrieve all doctors.
- getDoctor(): Method to get a specific doctor.
- addNewDoctor(): Method to add a new doctor.
- updateDoctor(): Method to update doctor details.
- deleteDoctor(): Method to delete a doctor.

6. Speciality Object

Attributes:

- speciality_id: Unique identifier for the speciality.
- speciality_name: Name of the speciality.

Methods:

- getAllSpecialties(): Method to retrieve all specialties.

7. Doctor Availability Object

Attributes:

- availability_id: Unique identifier for the doctor's availability.
- doctor_id: ID of the doctor.
- date: Date for availability.
- start_time: Start time of availability.
- end_time: End time of availability.
- is_delete: Boolean indicating if the availability record is deleted.

Methods:

- addDoctorAvailability(): Method to add a new availability for a doctor.
- removeDoctorAvailability(): Method to remove availability.
- updateAvailability(): Method to update availability details.
- fetchAvailability(): Method to retrieve availability for a doctor.
- fetchAllBySpeciality(): Method to retrieve all availability for a speciality.
- fetchTimeSlots(): Method to get available time slots.

Relationships:

1. Patients and Appointment
   - Relationship: Many-to-One
   - Explanation: A single patient can have multiple appointments, but each appointment is associated with only one patient. This is represented by the patient_id attribute in the Appointment class, linking it to the Patients class.

2. Doctor and Appointment
   - Relationship: Many-to-One
   - Explanation: A doctor can have multiple appointments with different patients, but each appointment is associated with only one doctor. The Appointment class has a doctor_id attribute to establish this link to the Doctor class.

3. Facility and Appointment
   - Relationship: Many-to-One
   - Explanation: Multiple appointments can take place in the same facility, but each appointment is associated with only one facility. This is represented by the facility_id attribute in the Appointment class, connecting it to the Facility class.

4. Doctor and Facility
   - Relationship: Many-to-Many
   - Explanation: A doctor can be available at multiple facilities, and a facility can host multiple doctors. The Doctor Availability class serves as an intermediary to establish this many-to-many relationship by associating doctors with facilities based on their availability.

5. Speciality and Doctor
   - Relationship: Many-to-Many
   - Explanation: Many doctors can have the same speciality, but each doctor is associated with many speciality. This is represented by the speciality_id attribute in the Doctor class, linking it to the Speciality class.

6. Speciality and Facility
   - Relationship: Many-to-Many
   - Explanation: A facility can offer multiple specialties, and each speciality can be offered by multiple facilities. This many-to-many relationship is managed through the specialization attribute in the Facility class, which holds an array of speciality IDs.

7. Doctor and Doctor Availability
   - Relationship: One-to-Many
   - Explanation: A doctor can have multiple availability records (different dates and times), but each availability entry is associated with only one doctor. The Doctor Availability class has a doctor_id attribute linking it to a single doctor.

8. Facility and Doctor Availability
   - Relationship: One-to-Many
   - Explanation: A facility can have multiple availability records for different doctors, but each availability entry links to only one facility, representing the times a doctor is available at that specific location.

9. Speciality and Doctor availability
   - Relationship: Many-to-Many
   - Explanation: Multiple doctors with the same specialty can have separate availability entries, and each availability can link to multiple specialties, allowing flexibility in matching doctor availability to patient needs.

Database Model

Patient Schema:

```
const PatientSchema = new mongoose.Schema({
   full_name: { type: String, required: true },
   email: { type: String, required: true, unique: true },
   password: { type: String, required: true },
   gender: { type: String, enum: ['Male', 'Female', 'Other'], required: true },
   dob: { type: Date, required: true },
   credit_card_no: { type: String, required: true },
   credit_card_exp_date: { type: Date, required: true },
   credit_card_cvv: { type: String, required: true },
   zip_code: { type: String, required: true },
});
```

Doctor Schema :

```
const DoctorSchema = new mongoose.Schema({
   name: { type: String, required: true },
   specialty: [String],
   email: { type: String, required: true, unique: true },
   degree: String,
   experience: String,
   password: String, // This should be hashed in the future
   isDeleted: { type: Boolean, default: false } // Soft delete field
});
```

Doctor Availability Schema:

```
const DoctorAvailabilitySchema = new mongoose.Schema({
   availability_date: { type: Date, required: true },
   doctor_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Doctor', required: true },
   start_time: { type: Date, required: true },
   end_time: { type: Date, required: true },
   facility_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Facility', required: true },
   speciality_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Specialization', required: true },
   isDelete: { type: Boolean, default: false }
});
```

Admin Schema:

```
const AdminSchema = new mongoose.Schema({
   admin_user_id: { type: String, required: true, unique: true },
   password: { type: String, required: true }
```

```
});
```

Facilities schema:

```
const FacilitySchema = new Schema({
   name: { type: String, required: true, unique: true },
   rooms: [RoomSchema],
   isDeleted: { type: Boolean, default: false } // Soft delete flag
});
```

Specialization schema:

```
const SpecializationSchema = new mongoose.Schema({
   name: { type: String, required: true, unique: true }
});
```

Appointment Schema:

```
const AppointmentSchema = new mongoose.Schema({
   patient_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Patient', required: true },
   facility_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Facility', required: true },
   doctor_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Doctor', required: true },
   speciality_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Specialization', required: true },
   appointment_date: { type: Date, required: true },
   appointment_scheduled_duration: { type: Number, required: true },
   appointment_original_duration: { type: Number },
   appointment_start_time: { type: Date, required: true },
   medical_diagnose: { type: String },
   medical_history: { type: String },
   current_signs: { type: String },
   additional_notes: { type: String },
   isComplete: { type: Boolean, default: false },
   isDelete: { type: Boolean, default: false }
});
```

Inter-Process Communication

The WeCureIT system utilizes RESTful APIs to enable seamless communication between the React-based frontend, Node.js/Express backend, and the MongoDB database. This architecture ensures efficient, scalable, and reliable data exchange while adhering to stateless communication principles.

Communication Protocols

- **REST (Representational State Transfer)**
  REST forms the backbone of communication in the WeCureIT system. It ensures standardized and scalable interactions between the client and server.

  Key HTTP Methods used:
  GET: Fetches data, such as a list of available doctors or facilities.
  POST: Sends data to the server, such as sending user data to create new accounts.
  PUT: Updates existing resources, such as modifying patient details.
  DELETE: Deletes resources, such as canceling an appointment.

- **Stringification (Serialization):**
  This occurs when the backend (Node.js/Express) sends a JavaScript object or MongoDB document as a JSON string to the frontend.

- **Parsing (Deserialization):**
  This occurs when the backend receives JSON data from the frontend and converts it into a usable JavaScript object.

Software Functionality

Patient user flow

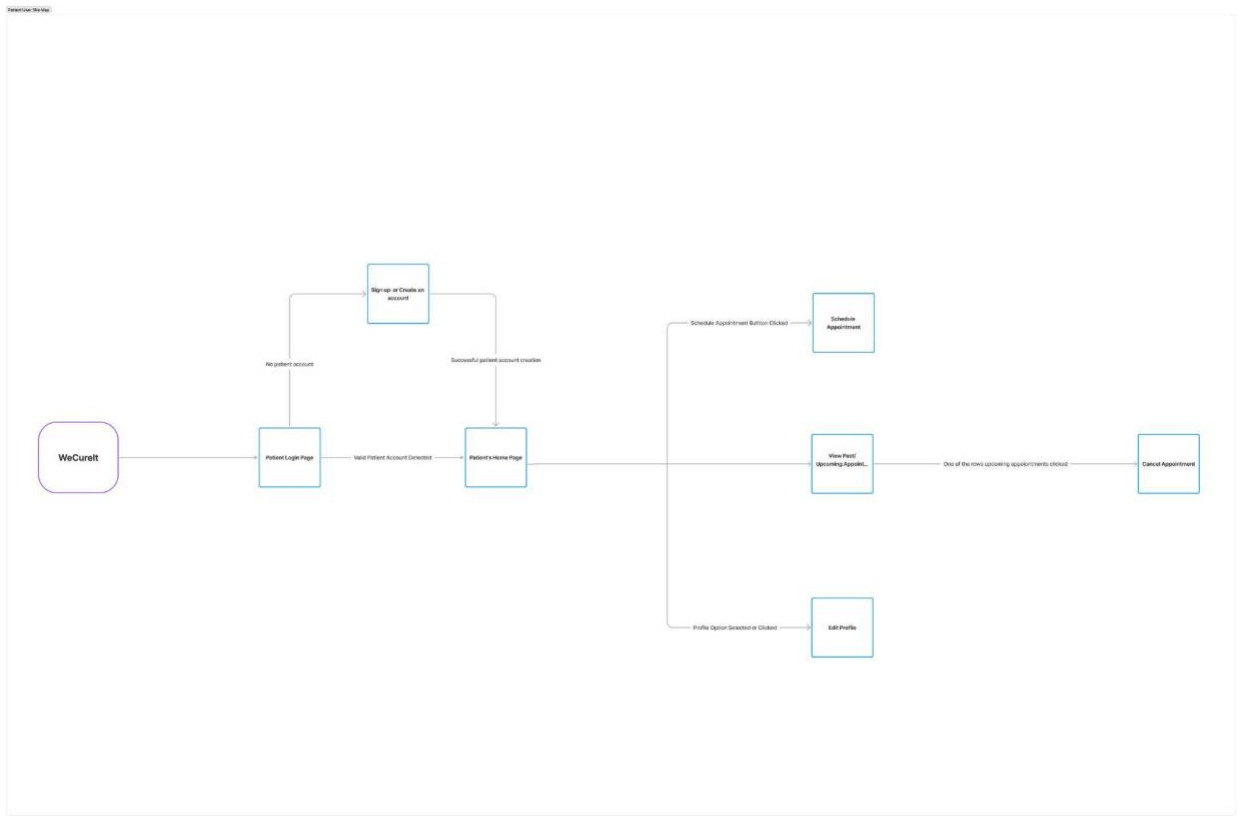

Figure 3 : *General outline of patient's navigation*

Admin user flow

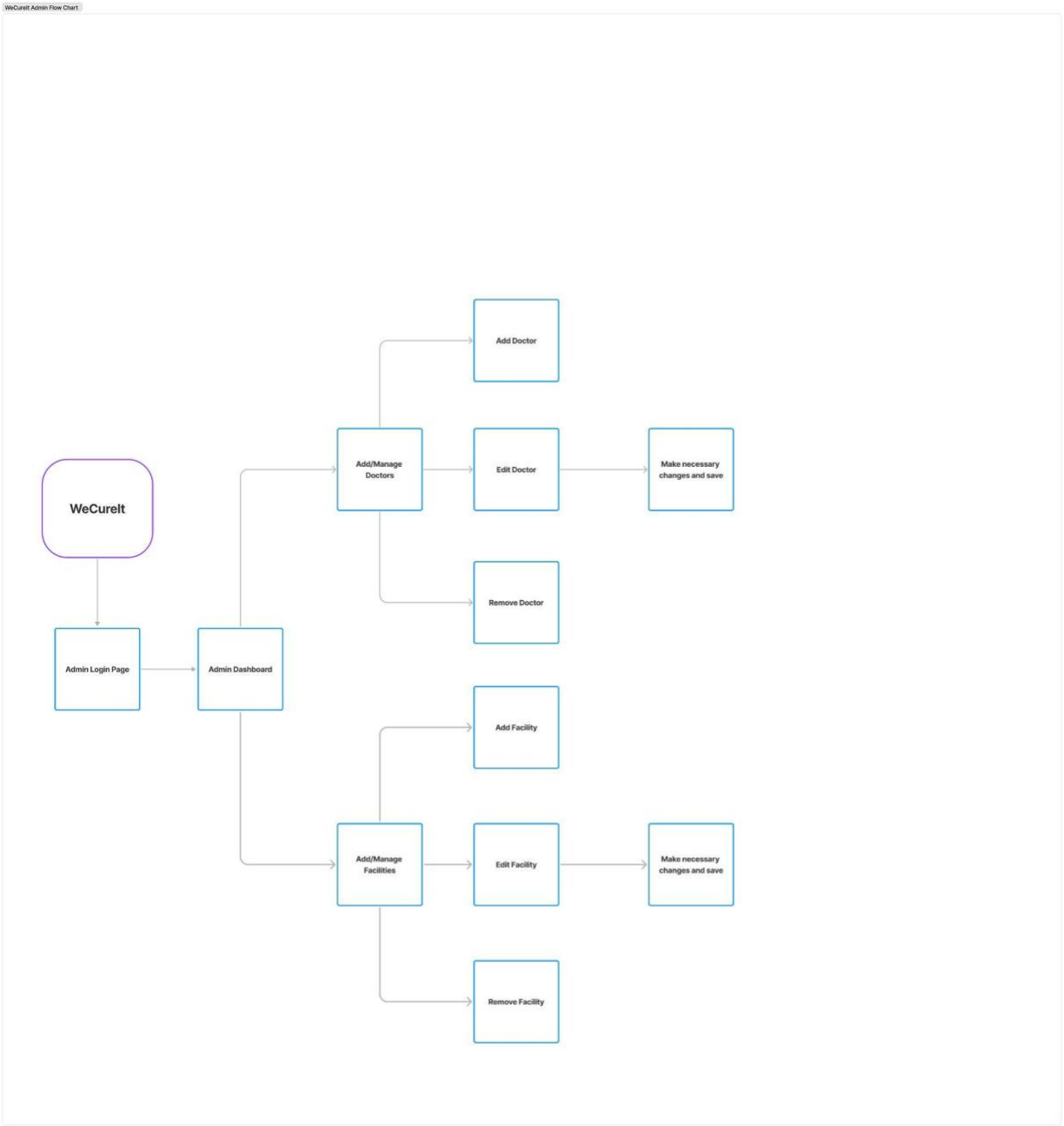

Figure 4: *General outline of admin's navigation*

Doctor user flow

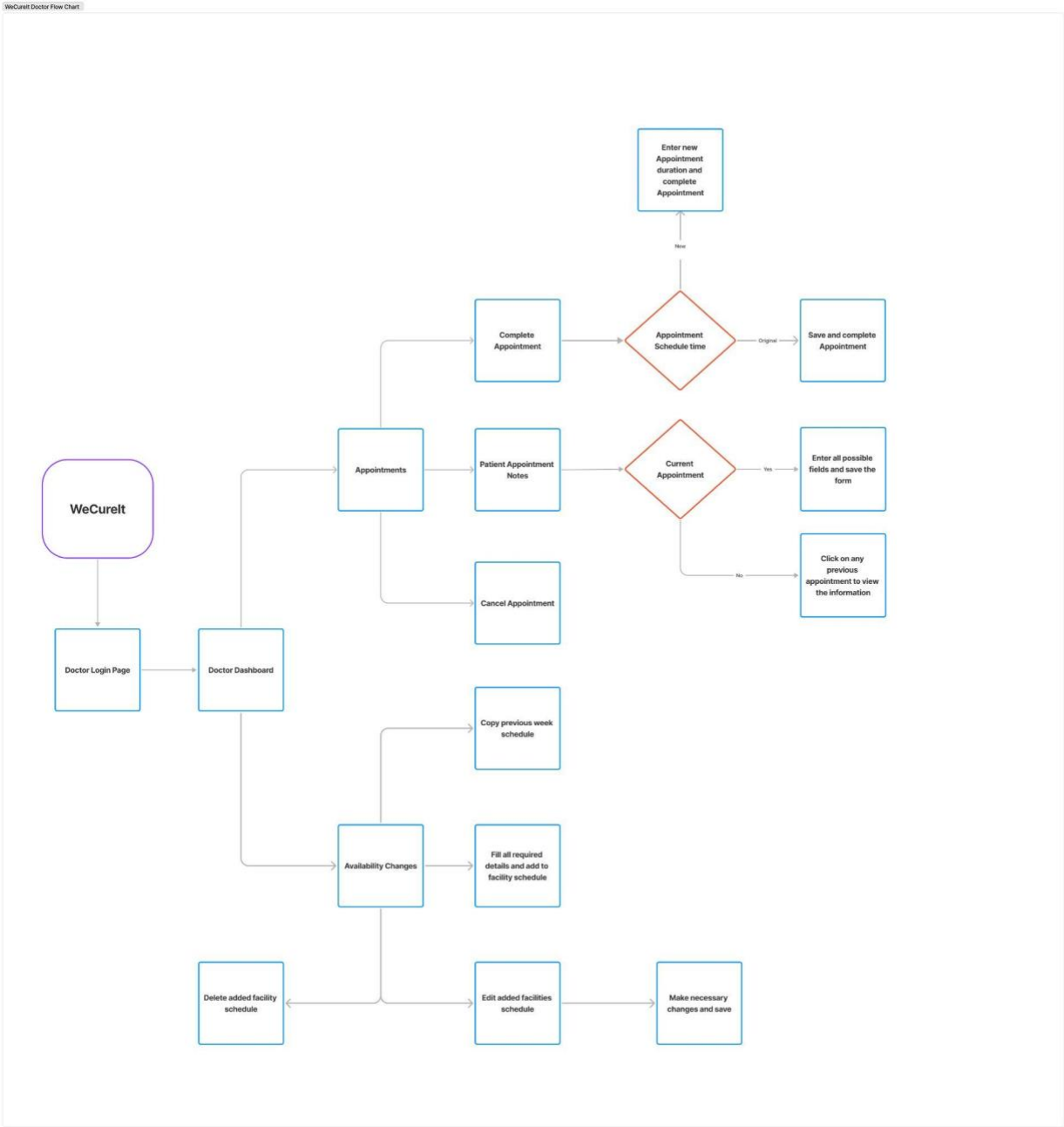

Figure 5: *General outline of doctor's navigation*

Patient mode:

Function Execution

Patient: Login to WeCureIt

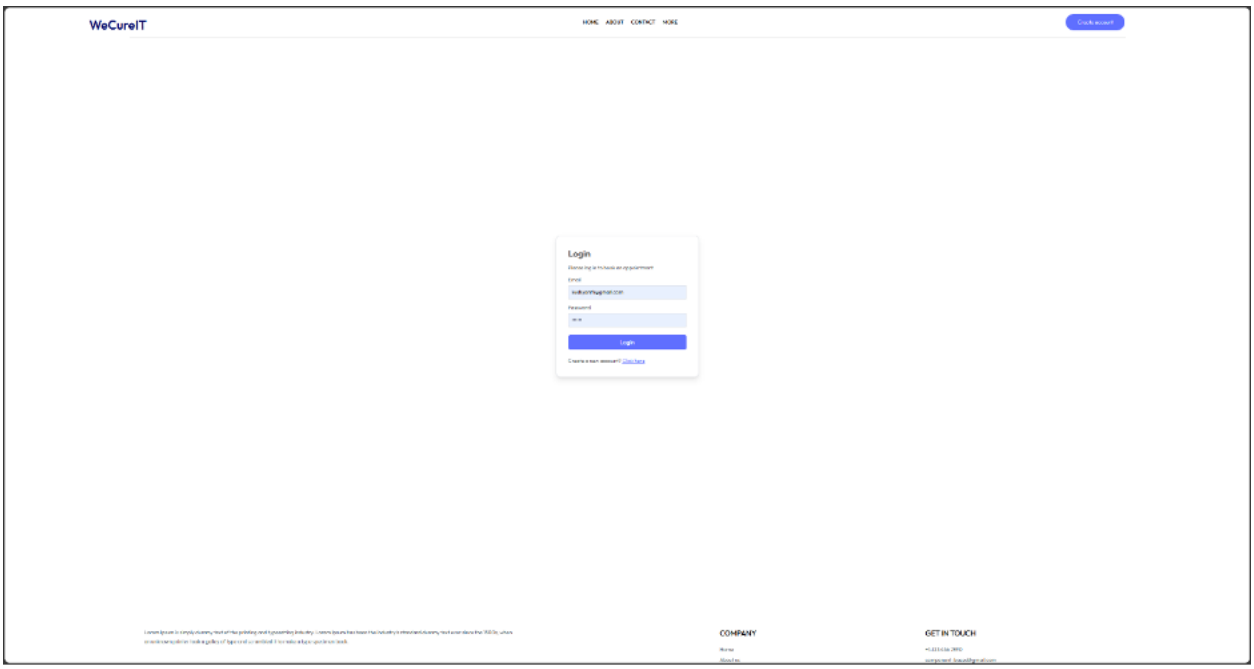

Figure 6: *Frontend: WeCureIt login page*

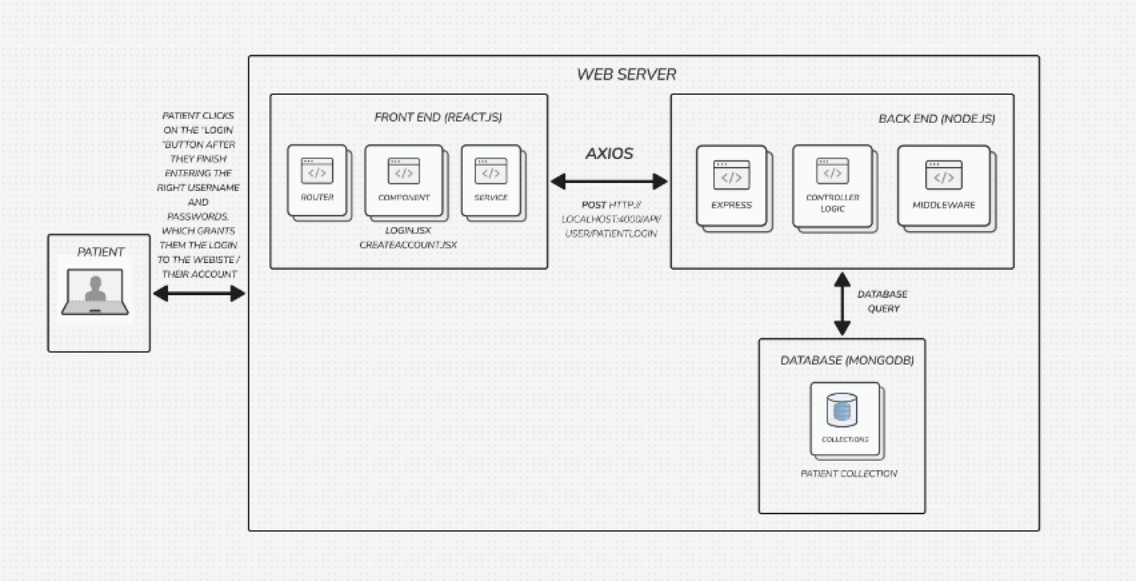

Figure 7*: Diagram: Retrieval of Patients credentials*

The patient needs to provide their email address and password in order to log in. As seen in Figure 1, this data is entered into the frontend that is powered by React. In the current system,

there is a dedicated login page exclusively for patients, with separate login pages for doctors and administrators. This design allows for a streamlined and efficient login process by focusing each login page specifically on one type of user, which simplifies the validation process and improves performance. For the patient login page, only one API call is necessary to validate credentials against the Patient table. This dedicated API request checks the patient's email and password, ensuring that only patients are validated on this page. Since there’s no need to validate other user types, this approach reduces database queries and minimizes server load.

If the API call for a given login page does not locate a user with matching credentials, an error message stating "Invalid login credentials" is displayed to the user. However, if the credentials are verified, the user is directed to the appropriate home page for their role.

It’s worth noting that there are only two possible outcomes for any login attempt: either the API call successfully finds a match, or it fails entirely. This is because each user's email address is unique within their respective user role tables (Patient, Doctor, Admin), ensuring no overlap across different user types.

Patient: Create an account for WeCureIt

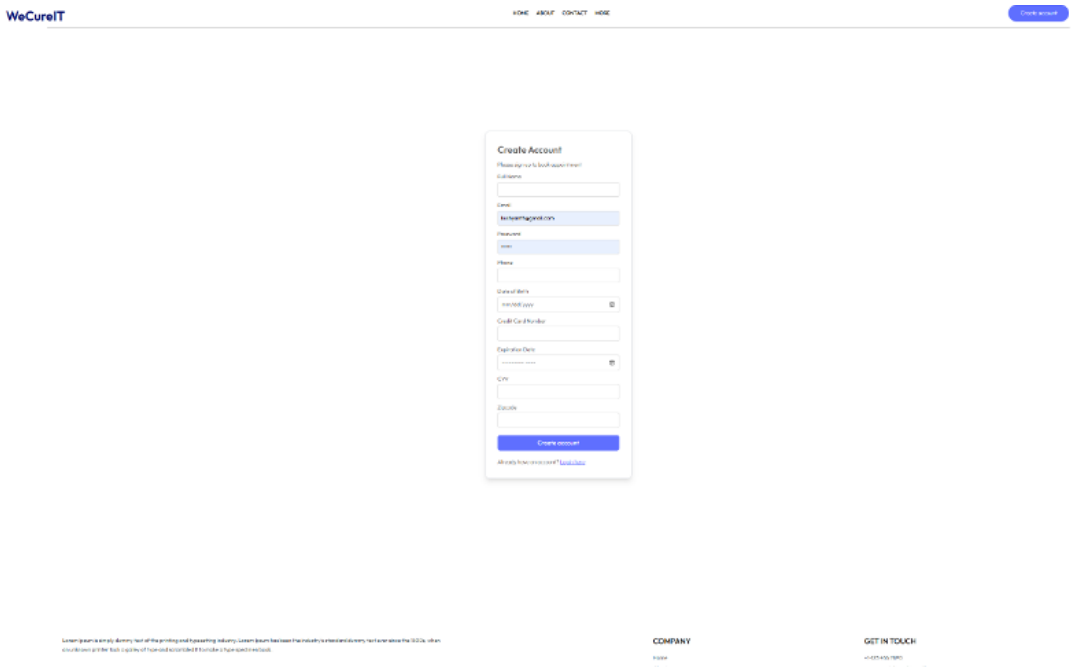

Figure 8: *Frontend: WeCureIt sign up page*

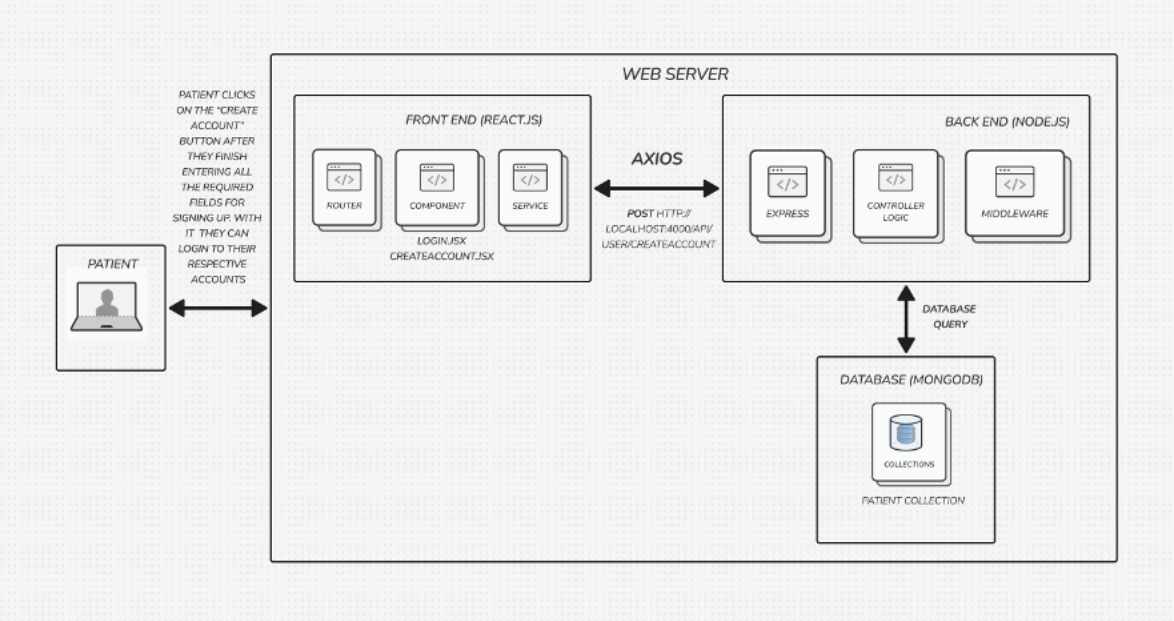

Figure 9: *Diagram: Creation of a new patient account*

WeCureIt requires patients to have an account in order to access any of its features. They must successfully complete the two screens (Figures 6 and 7) to create an account if they don't already have one. Additionally, React is used on both screens. The patient must put in all required information in Figure 6. Upon clicking the "Create an Account" button, an API POST request is made in order to store all of the data gathered from the two pages in the appropriate tables. The Users table is modified when a patient creates an account. If the email address provided is unique, the Users table is updated to store the patient's information, including contact and login details. Unlike other cases where separate tables might exist for payment and preferences, our system does not split this data across multiple tables. Instead, additional details, such as patient preferences, can be updated at a later time directly within the Users table or through associated fields once provided by the patient. This approach centralizes patient information, simplifying data retrieval and account management.

Patient: Edit Profile, Payment Method and Address

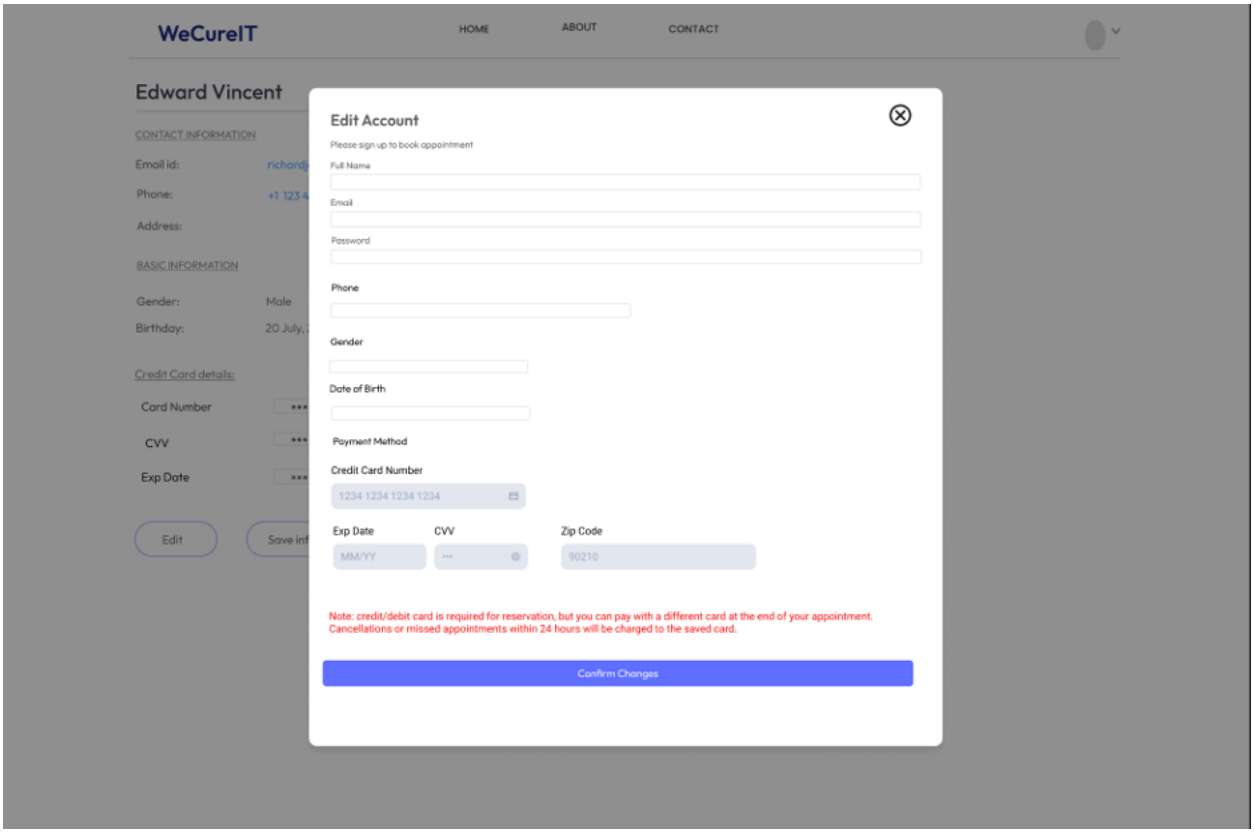

Figure 10: *Frontend: WeCureIt edit profile page*

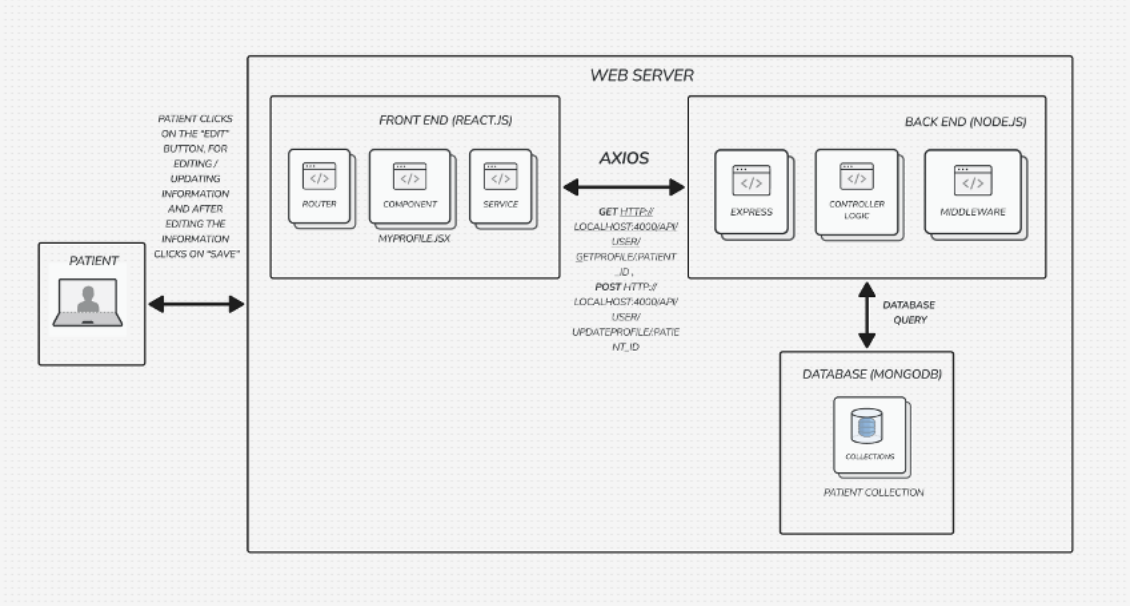

Figure 11: *Update profile information*

Patients have the flexibility to update their profile information—including personal details, payment methods, and address information—all within a single page, as shown in Figure 5. When the patient makes changes in the frontend and clicks the "Save Changes" button, an Axios HTTP POST request is sent to update the relevant fields. Unlike segmented forms, our system allows users to edit all essential information in one streamlined interface, enabling them to review and modify their personal, billing, and address details simultaneously. The Users table will then be updated to reflect any modifications made, ensuring that all patient information is accurate and up-to-date in one cohesive update.

Patient: Schedule an Appointment

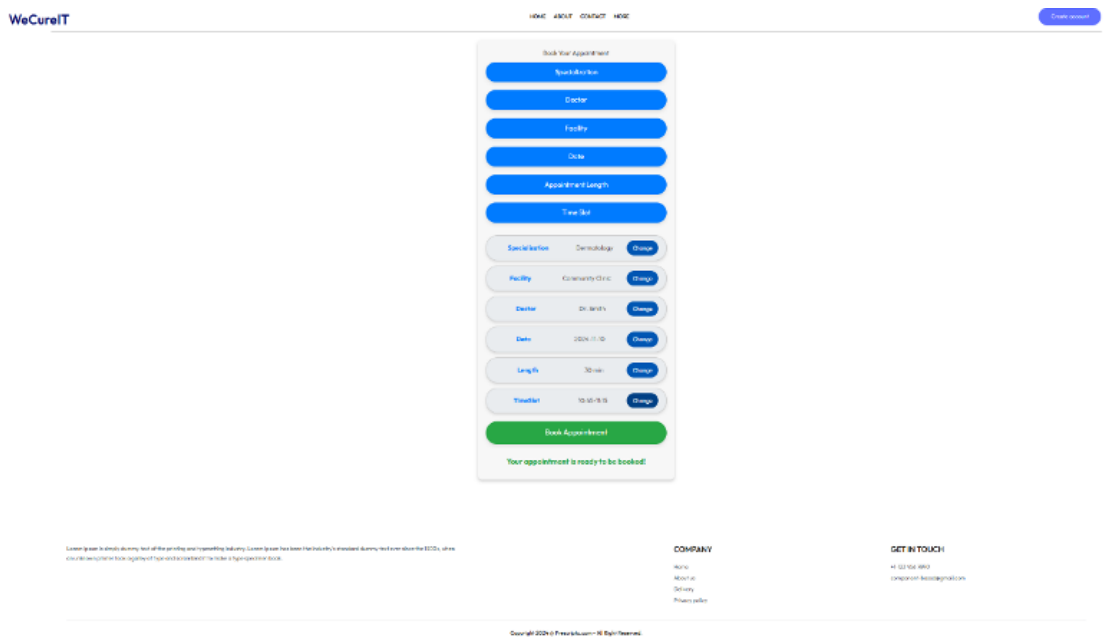

Figure 12: *Frontend: WeCureIt appointment booking page*

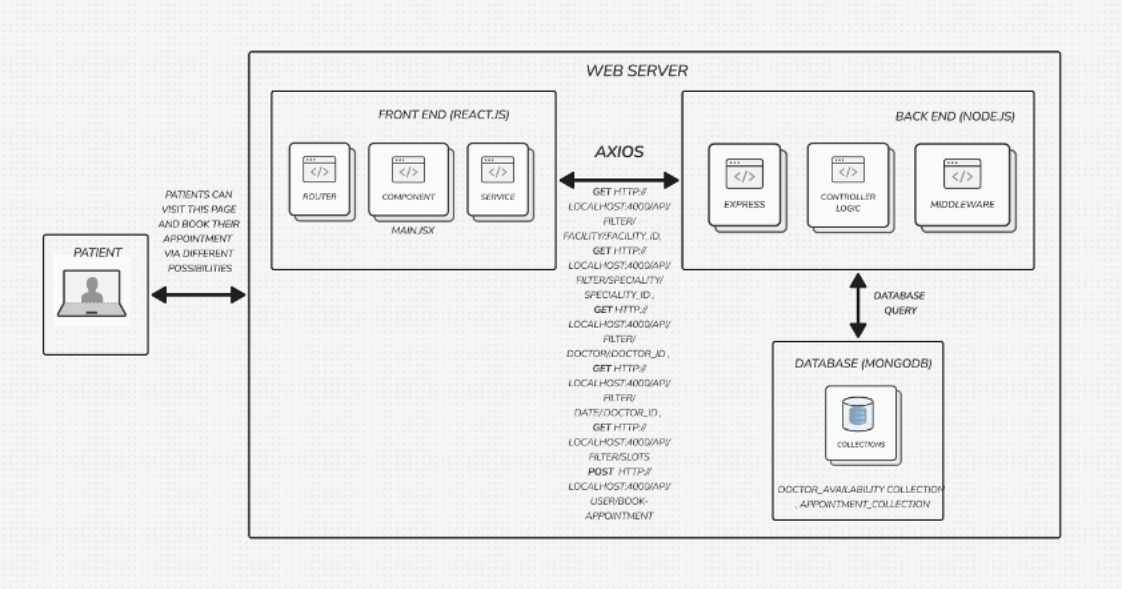

Figure 13: *Scheduling an appointment*

Patients can book an appointment in many ways, following are the four main possible ways.

Case #1: Patient selects a specific speciality:

1. Patient provides the following initial input (required):

a) Specialization (selected from a list of specializations available in the system)

2. Fetching available doctors and facilities:

a) Using the selected specialization ID, query the **Doctor** table to retrieve doctors associated with this specialization.

b) Simultaneously, query the **Facility** table to retrieve facilities that offer services in the selected specialization.

3. Patient selects from filtered options:

a) Display the list of doctors (fetched in Step 2) in a dropdown menu.

b) Display the list of facilities (fetched in Step 2) in a dropdown menu.

c) Display the list of date (fetched in Step 2) in a dropdown menu.

d) Allow the patient to select a doctor and a facility.

4. Optional date selection:

a) Allow the patient to choose a preferred date for the appointment.

b) If a date is selected:

a. Query the **Doctor_Schedule** table using the doctor ID to fetch available slots for that date.

c) If no date is selected:

a. All available dates and slots are calculated based on the doctor's schedule.

5. Calculate available time slots:

a) Retrieve all existing appointments for the selected doctor and date (if provided) from the **Appointment** table.

b) Using the doctor's available hours (visiting_hours_start and visiting_hours_end) in **Doctor_Schedule**, calculate open time slots based on the selected appointment length.

c) Exclude time slots that overlap with existing appointments and add a mandatory 10-minute gap after each.

d) Insert a mandatory 30-minute break after every 4 hours of work for the doctor.

e) Stop calculations at the doctor's end time for that day.

6. Finalize selection:

a) Based on the calculated time slots in Step 5, allow the patient to select an available time slot that best fits their needs.

b) If the date was not selected earlier, prompt the patient to choose from the available dates.

7. Confirm booking:

a) Store the finalized booking details, including specialization, doctor, facility, date, time, and appointment length, in the **Appointment** table.

b) Display a confirmation message or appointment summary to the patient.

Case #2: Patient selects a specific Facility:

1. Patient provides the following initial input (Required):

a) Facility (selected from a list of available facilities in the system)

2. Fetching available specializations and doctors:

a) Using the selected facility ID, query the **Doctor_Availability** table to retrieve specializations associated with this facility.

b) Query the **Doctor** table to retrieve doctors associated with the selected facility.

3. Patient selects from filtered options:

a) Display the list of specializations (fetched in Step 2) in a dropdown menu.

b) Display the list of doctors (fetched in Step 2) in a dropdown menu.

c) Display the list of date (fetched in Step 2) in a dropdown menu.

d) Allow the patient to select a specialization and a doctor.

4. Optional date selection:

a) Allow the patient to choose a preferred date for the appointment.

b) If a date is selected:

a. Query the **Doctor_Schedule** table using the selected doctor ID and date to fetch available slots for that date.

c) If no date is selected:

a. All available dates and slots are calculated based on the doctor's schedule.

5. Calculate available time slots:

a) Retrieve all existing appointments for the selected doctor and date (if provided) from the **Appointment** table.

b) Using the doctor's available hours (visiting_hours_start and visiting_hours_end) in **Doctor_Schedule**, calculate open time slots based on the selected appointment length.

c) Exclude time slots that overlap with existing appointments and add a mandatory 10-minute gap after each.

d) Insert a mandatory 30-minute break after every 4 hours of work for the doctor.

e) Stop calculations at the doctor's end time for that day.

6. Finalize selection:

a) Based on the calculated time slots in Step 5, allow the patient to select an available time slot that best fits their needs.

b) If the date was not selected earlier, prompt the patient to choose from the available dates.

7. Confirm booking:

1. Store the finalized booking details, including facility, specialization, doctor, date, time, and appointment length, in the **Appointment** table.
2. Display a confirmation message or appointment summary to the patient.

Case #3: Patient selects a specific Doctor:

1. Patient provides the following initial input (Required)**:**

a) Doctor (selected from a list of available doctors in the system)

2. Fetching available facilities and specializations:

a) Using the selected doctor ID, query the **Doctor_Availability** table to retrieve facilities associated with this doctor.

b) Query the **Specialization** table to retrieve specializations associated with the selected doctor.

3. Patient selects from filtered options:

a) Display the list of facilities (fetched in Step 2) in a dropdown menu.

b) Display the list of specializations (fetched in Step 2) in a dropdown menu.

c) Display the list of date (fetched in Step 2) in a dropdown menu.

d) Allow the patient to select a facility and a specialization.

4. Optional date selection:

a) Allow the patient to choose a preferred date for the appointment.

b) If a date is selected:

a. Query the **Doctor_Schedule** table using the selected doctor ID and date to fetch available slots for that date.

c) If no date is selected:

a. All available dates and slots are calculated based on the doctor's schedule.

5. Calculate available time slots:

a) Retrieve all existing appointments for the selected doctor and date (if provided) from the **Appointment** table.

b) Using the doctor's available hours (visiting_hours_start and visiting_hours_end) in **Doctor_Schedule**, calculate open time slots based on the selected appointment length.

c) Exclude time slots that overlap with existing appointments and add a mandatory 10-minute gap after each.

d) Insert a mandatory 30-minute break after every 4 hours of work for the doctor.

e) Stop calculations at the doctor's end time for that day.

6. Finalize selection:

a) Based on the calculated time slots in Step 5, allow the patient to select an available time slot that best fits their needs.

b) If the date was not selected earlier, prompt the patient to choose from the available dates.

7. Confirm booking:

a) Store the finalized booking details, including doctor, specialization, facility, date, time, and appointment length, in the **Appointment** table.

b) Display a confirmation message or appointment summary to the patient.

Case #4: Patient selects a specific Date:

1. Patient provides the following initial input (Required)**:**

a) Date (chosen from a date picker)

2. Fetching available specializations, facilities, and doctors:

a) Using the selected date, query the **Doctor_Availability** table to retrieve all doctors who are available on that date.

b) From the doctor availability data:

a. Retrieve associated **specialization IDs**.

b. Retrieve associated **facility IDs**.

c) Query the **Specialization** and **Facility** tables to retrieve names for each of the unique specialization and facility IDs obtained in Step 2.

3. Patient selects from filtered options:

a) Display the list of specializations (fetched in Step 2) in a dropdown menu.

b) Display the list of facilities (fetched in Step 2) in a dropdown menu.

c) Display the list of doctors (filtered by availability on the selected date) in a dropdown menu.

d) Allow the patient to select a specialization, facility, and doctor based on availability for the chosen date.

4. Optional appointment length selection:

a) Allow the patient to select a preferred appointment length (e.g., 15 minutes, 30 minutes, or 1 hour).

5. Calculate available time slots:

a) Retrieve all existing appointments for the selected doctor and date from the **Appointment** table.

b) Using the doctor's available hours (visiting_hours_start and visiting_hours_end) for that date, calculate open time slots based on the selected appointment length.

c) Exclude time slots that overlap with existing appointments and add a mandatory 10-minute gap after each.

d) Insert a mandatory 30-minute break after every 4 hours of work for the doctor.

e) Stop calculations at the doctor's end time for that day.

6. Finalize selection:

a) Based on the calculated time slots in Step 5, allow the patient to select an available time slot that best fits their needs.

b) If a specific doctor, specialization, or facility was not selected earlier, prompt the patient to select from the remaining filtered options.

7. Confirm booking:

a) Store the finalized booking details, including date, specialization, facility, doctor, time, and appointment length, in the **Appointment** table.

b) Display a confirmation message or appointment summary to the patient

Patient: View Appointments

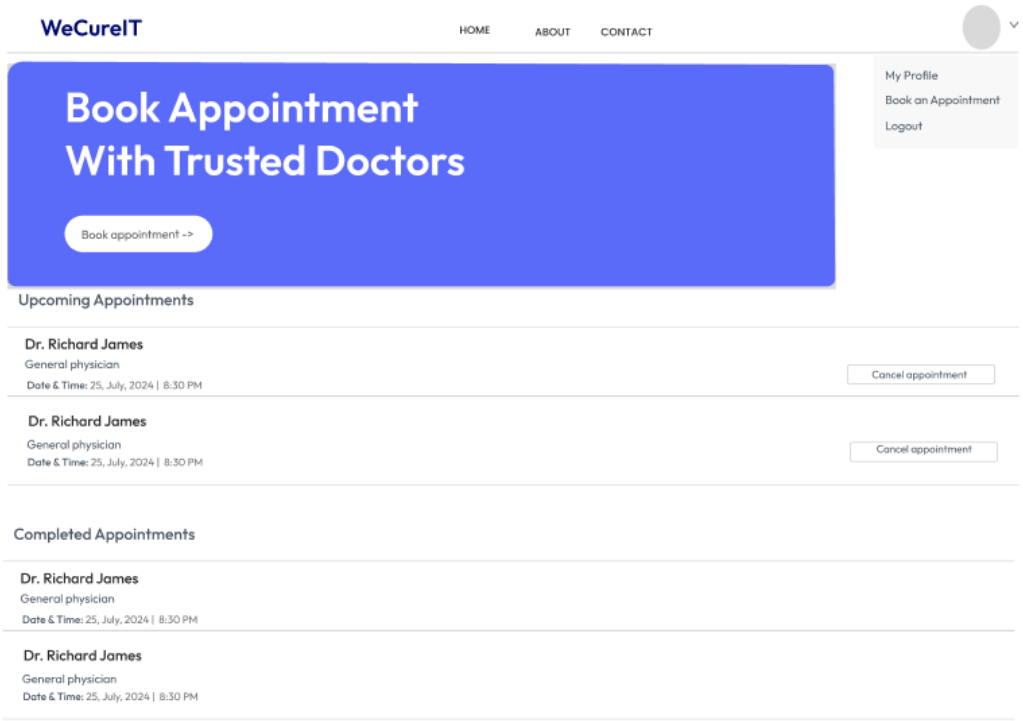

Figure 14: *Patients Appointment Page*

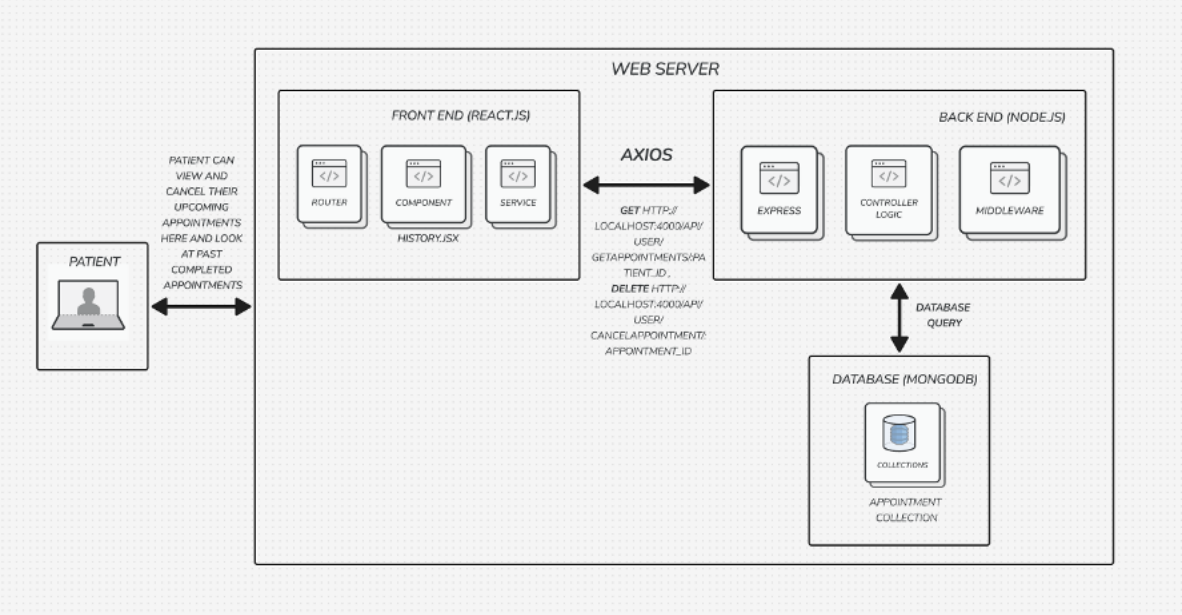

Figure 15: *View patient's appointments*

In our system, patients have the capability not only to view all appointments—past and upcoming—but also to manage them actively, including the option to cancel appointments if necessary. To display a patient's appointments, the system retrieves relevant records from the Appointment table by matching entries to the patient_id, which is tracked from the point of account creation or login. This data is then presented on the screen, as illustrated in Figure 9, allowing the patient to view their appointment history and upcoming bookings.

In addition to viewing, patients can select any upcoming appointment to cancel it, initiating a cancellation request that updates the Appointment table accordingly. This functionality provides patients with greater control over their bookings directly from the interface, allowing for a seamless experience in managing their healthcare schedule.

Admin mode:

Admin login

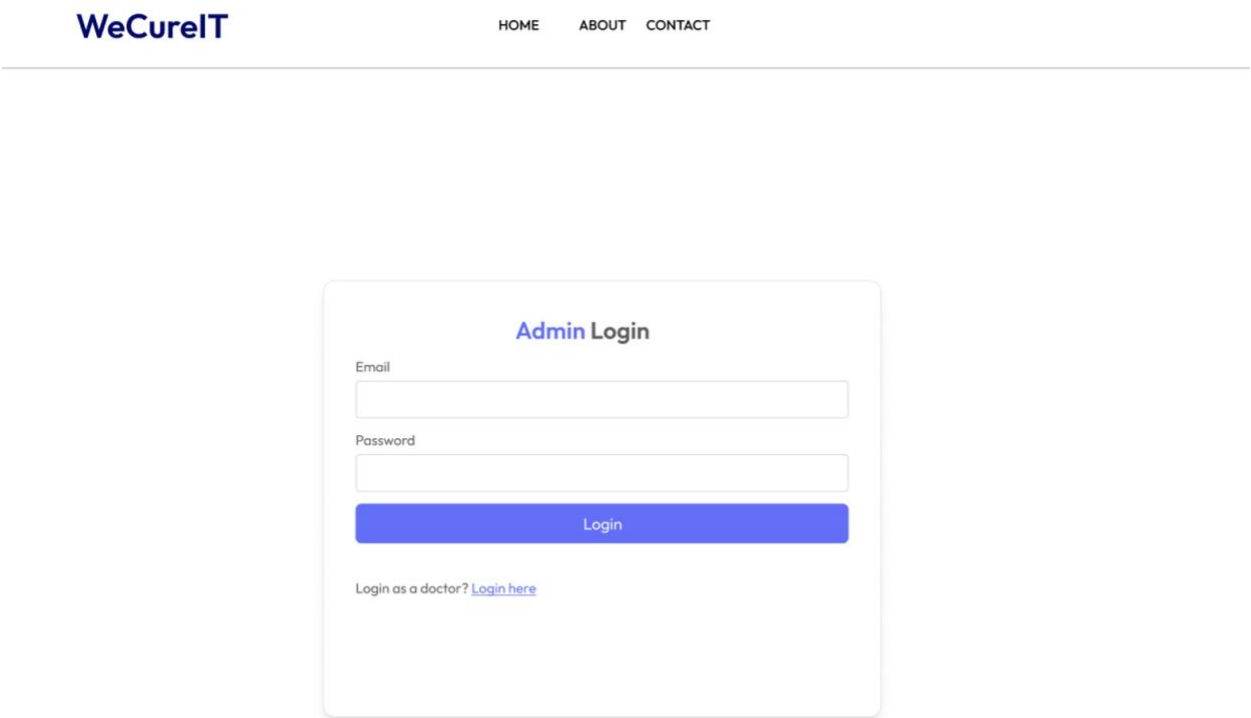

Figure 16 : *Admin login page*

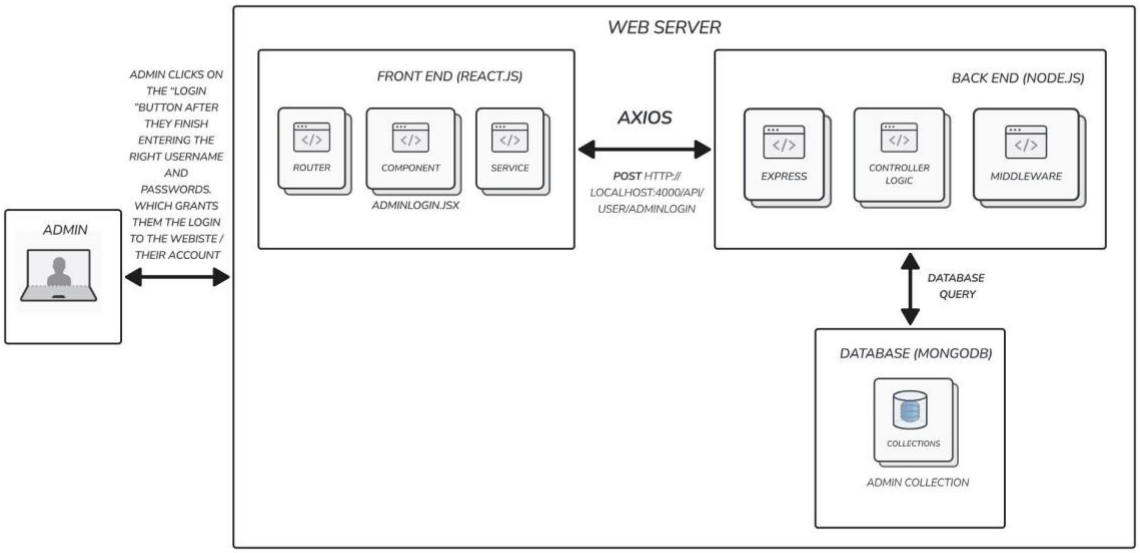

Figure 17*: Admin login*

In the "Admin Login" functionality, an administrator can securely access the system by entering their username and password on the React-based frontend login page. Upon clicking the "Login" button, the entered credentials are sent in a POST request via Axios to the backend endpoint, /api/admin/login.

The Node.js backend processes this request through controller logic and middleware, which validates the credentials and checks authorization. It then queries the Admin collection in MongoDB to verify the credentials. If the login is successful, the backend sends a confirmation response (such as a token or session data) back to the frontend, granting the admin access to their account. If authentication fails, an error message is displayed. This secure login process ensures that only authorized admins can access the system's administrative features.

Adding a new doctor

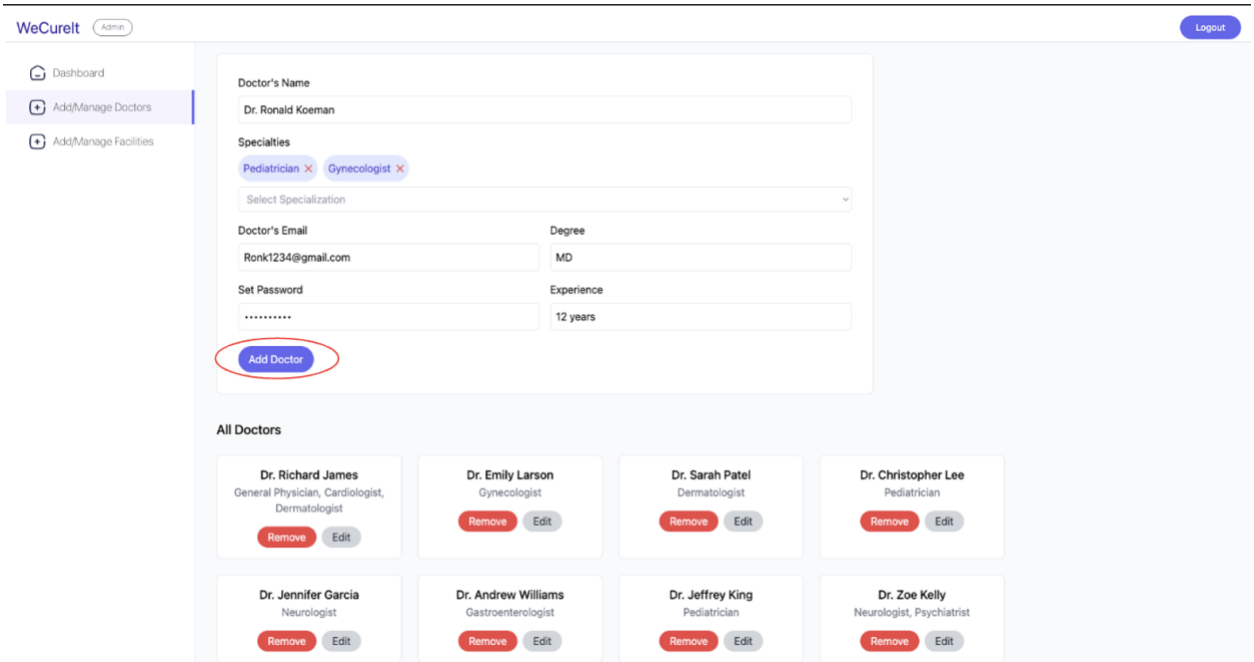

Figure 18: *The "Add Doctor" form in the admin interface after all information is filled.*

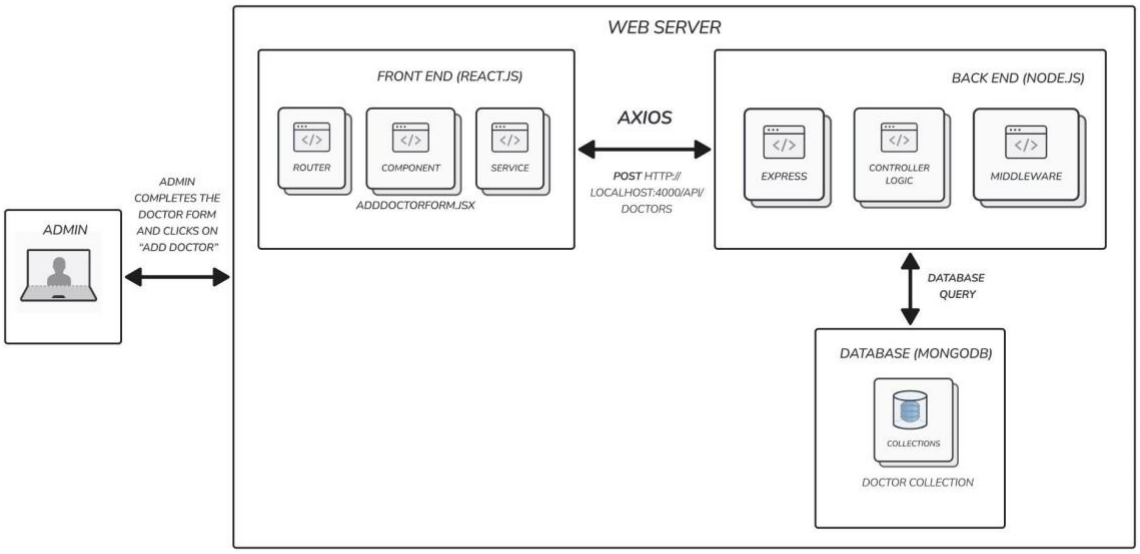

Figure 19: *Adding a new doctor to the Doctor's collection.*

In the "Add Doctor" functionality, an administrator has the ability to add new doctor profiles to the system via the frontend interface built with React. The admin accesses a dedicated form component where they enter the new doctor's details. Once the admin submits the form, a POST request is sent using Axios to a specific backend API endpoint, such as /api/doctors/add. This

request includes the doctor's data, which is then processed by the Node.js backend. In the backend, controller logic handles the incoming request.

The backend then interacts with the MongoDB database, where the new doctor's information is added as a document within the Doctor collection. If the insertion is successful, the backend returns a confirmation response to the frontend. This response triggers an update in the React interface, typically showing the new doctor in the list of doctors.

Removing a doctor

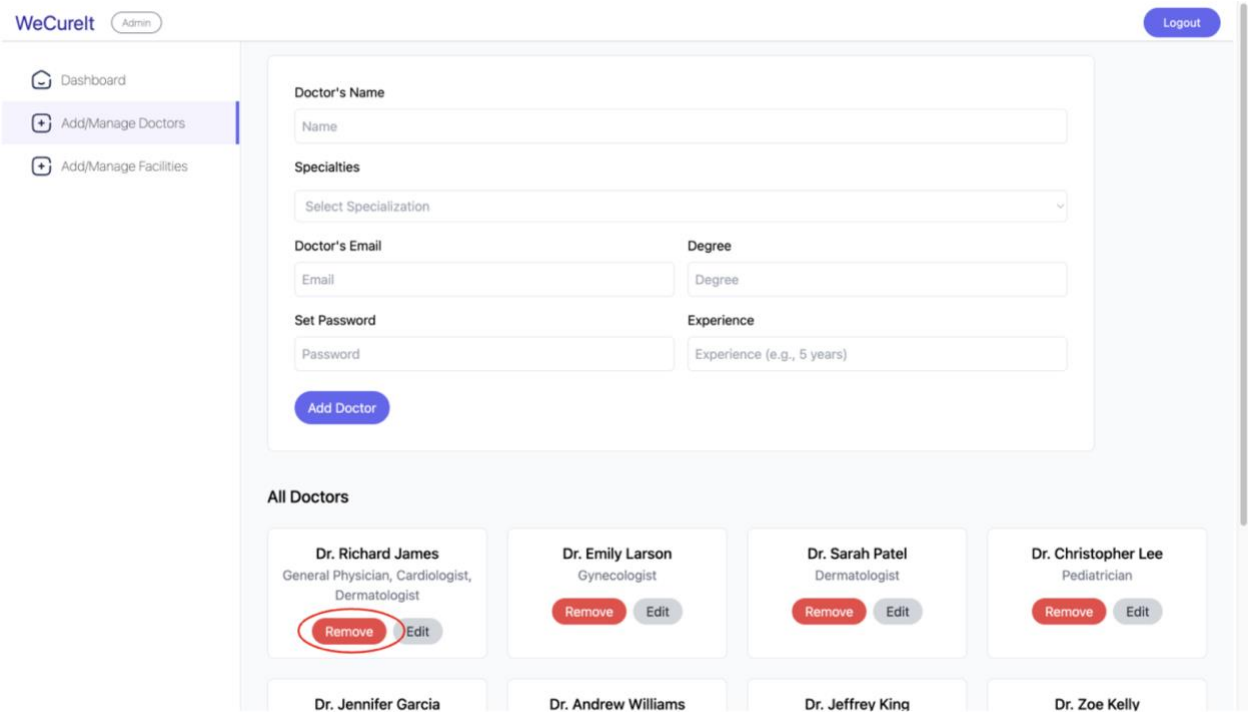

Figure 20: *The "Add/Manage Doctors" page showing the "Remove" button for a doctor.*

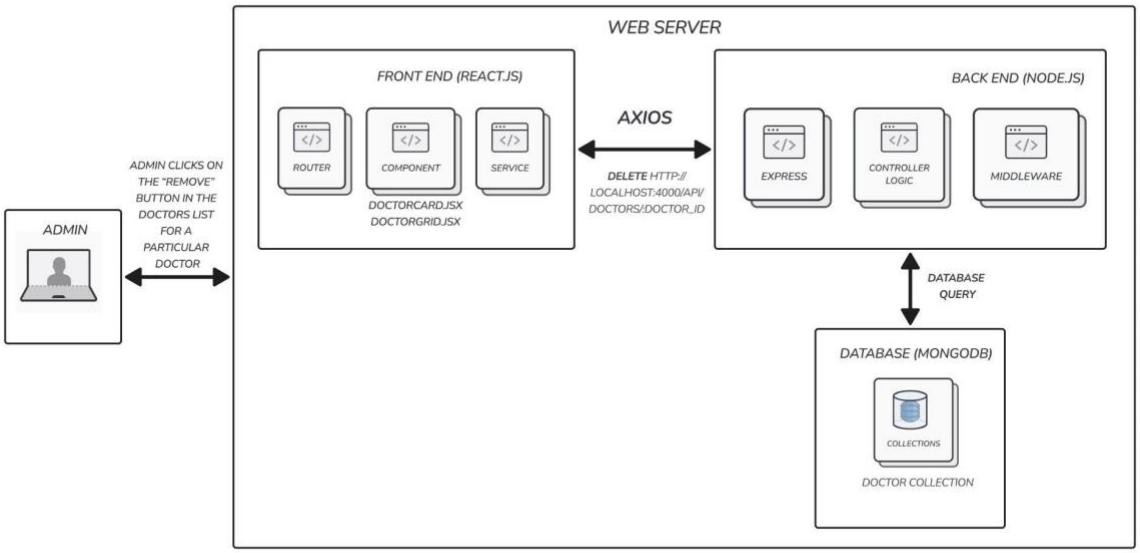

Figure 21: *Removing a doctor from the doctors collection.*

In the "Remove Doctor" functionality, an administrator shall soft delete a doctor's profile. The admin from the list of doctors, where each doctor has a "Remove" button. When the admin selects this option, a DELETE request is triggered via Axios to a backend API endpoint, such as /api/doctors/{doctor_id}, with the specific doctor's unique identifier included in the URL. This request is received by the Node.js backend, where controller logic handles the deletion operation.

Upon receiving the request, the backend interacts with the MongoDB database, soft deleting the document associated with the specified doctor ID in the doctors collection. If the deletion is successful, the backend responds with a confirmation to the frontend. This response prompts the React interface to update automatically, removing the doctor from the displayed list.

Edit doctor information

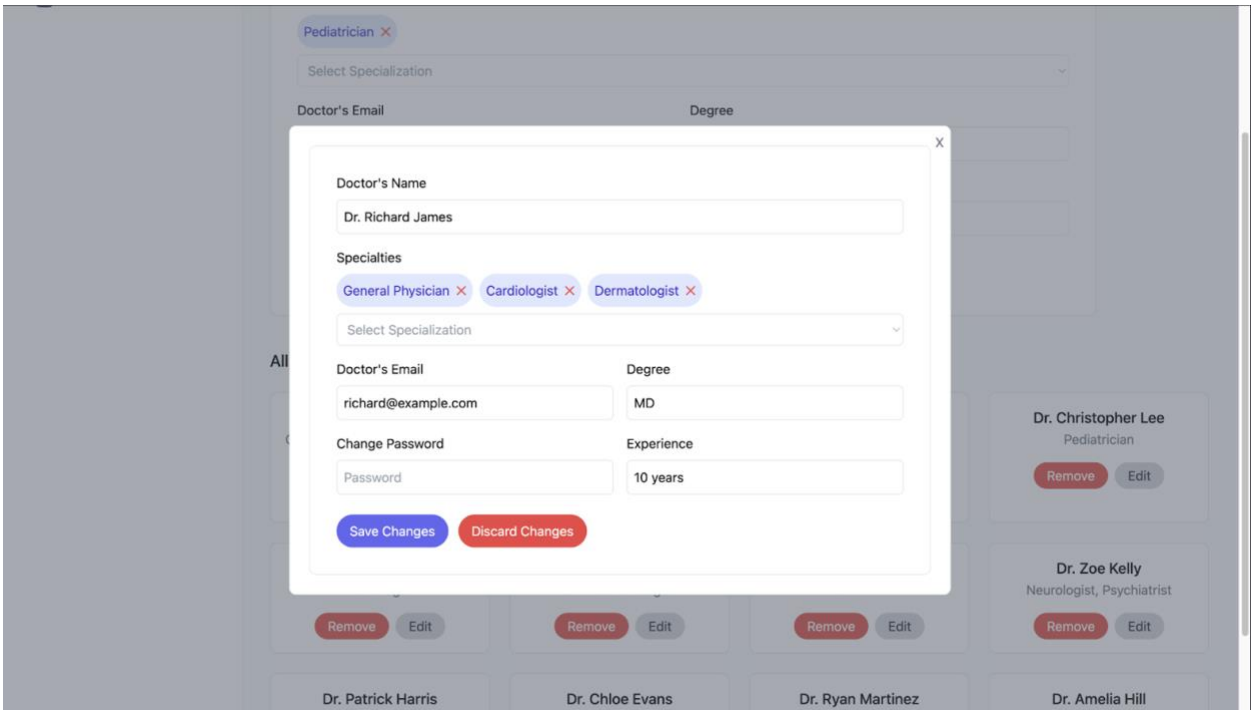

Figure 22: *The pop-up window that appears after clicking the "Edit" button for a doctor in the admin interface with current information.*

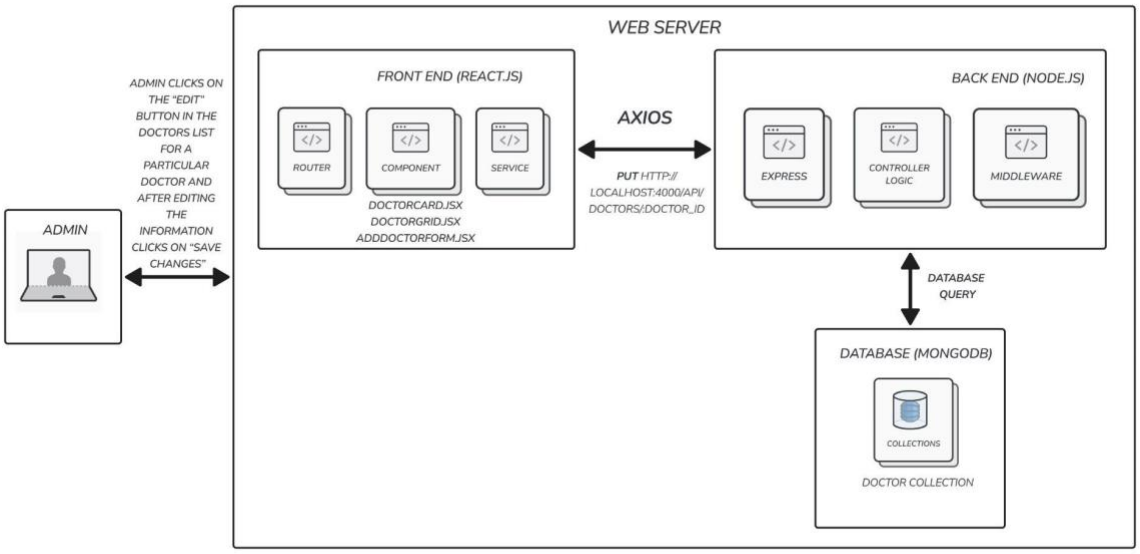

Figure 23: *Editing a doctor information in the doctors collection.*

In the "Edit Doctor Information" functionality, an administrator can update an existing doctor's profile through the React-based frontend. By clicking the "Edit" button next to a doctor's profile,

the admin opens a form pre-filled with the doctor's current information, such as name, specialization, and experience. After making changes, the admin submits the form, triggering a PUT request via Axios to a backend endpoint like /api/doctors/{doctor_id} with the updated data.

The Node.js backend processes this request, using controller logic and middleware to validate inputs and ensure authorized access. It then updates the doctor's record in the MongoDB Doctor collection. Upon a successful update, the backend sends a confirmation response to the frontend, which updates the doctor's information in the interface, reflecting the changes immediately for the admin.

Adding a facility

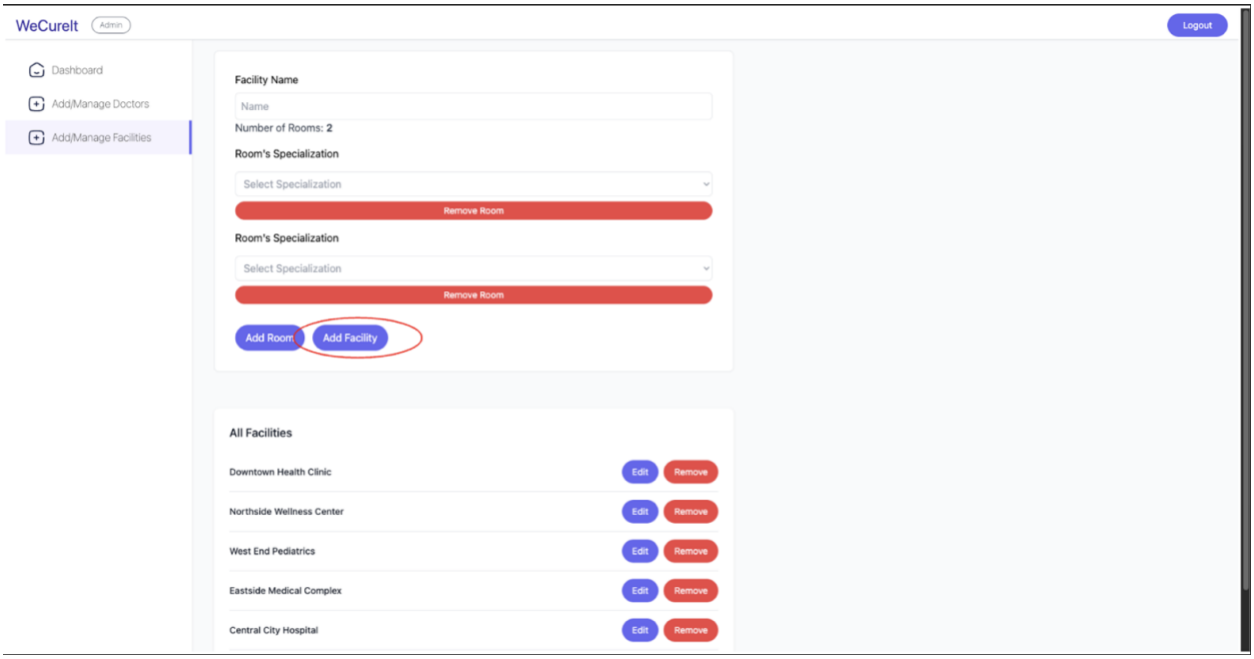

Figure 24 : *The "Add/Manage Facilities" page in the admin interface, showing the "Add Facility" button.*

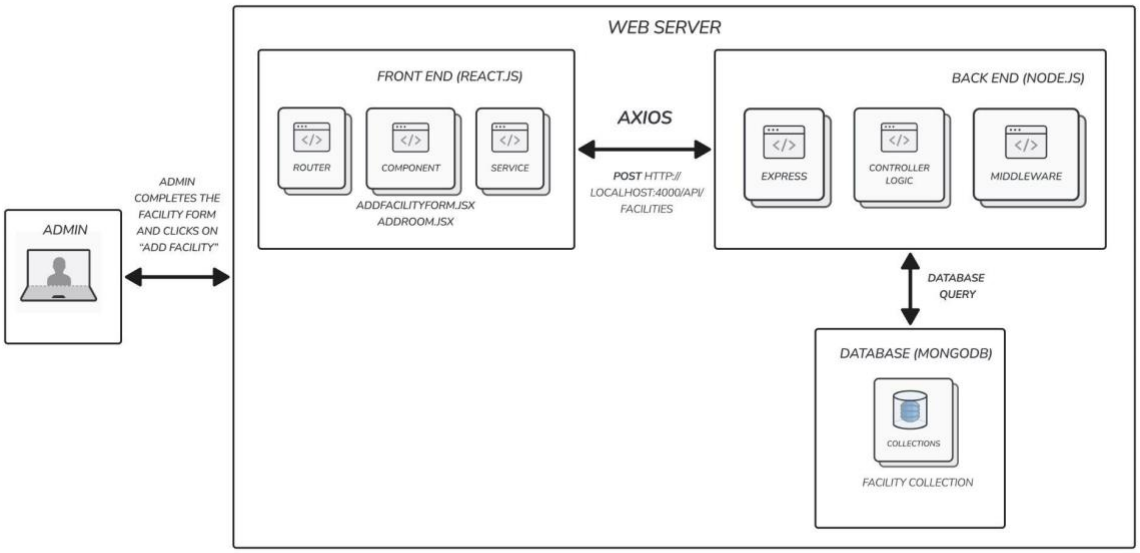

Figure 25*: Adding a new facility to the facilities collection.*

In the "Add Facility" functionality, an administrator can add new medical facilities to the system through a React-based frontend interface. The admin fills out a form with details about the facility, such as the name and any room specifications, and then submits the form by clicking "Add Facility." This action triggers a POST request via Axios to the backend endpoint, such as /api/facilities, with the facility data.

The Node.js backend receives this request. The backend then saves the new facility information in the MongoDB Facility collection. Once the database insertion is successful, the backend sends a confirmation response back to the frontend. The frontend then updates the interface, showing the newly added facility, allowing the admin to see the updated list immediately.

Removing a facility

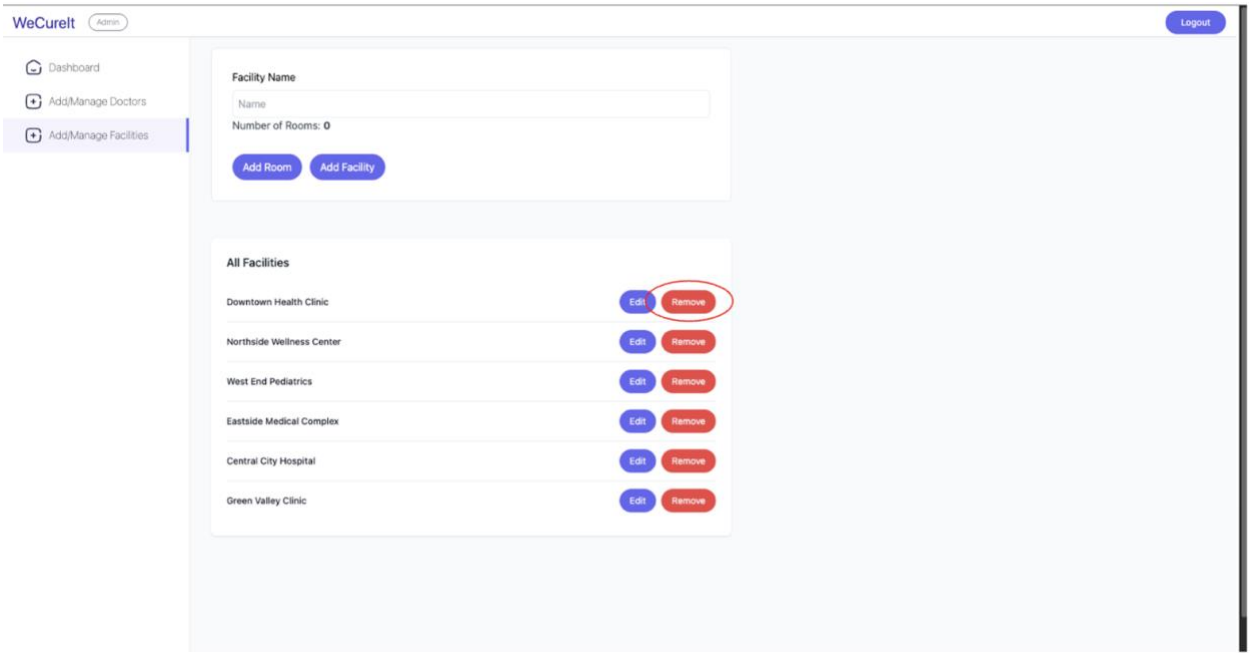

Figure 26: *The "Add/Manage Facilities" page showing the remove option for facilities.*

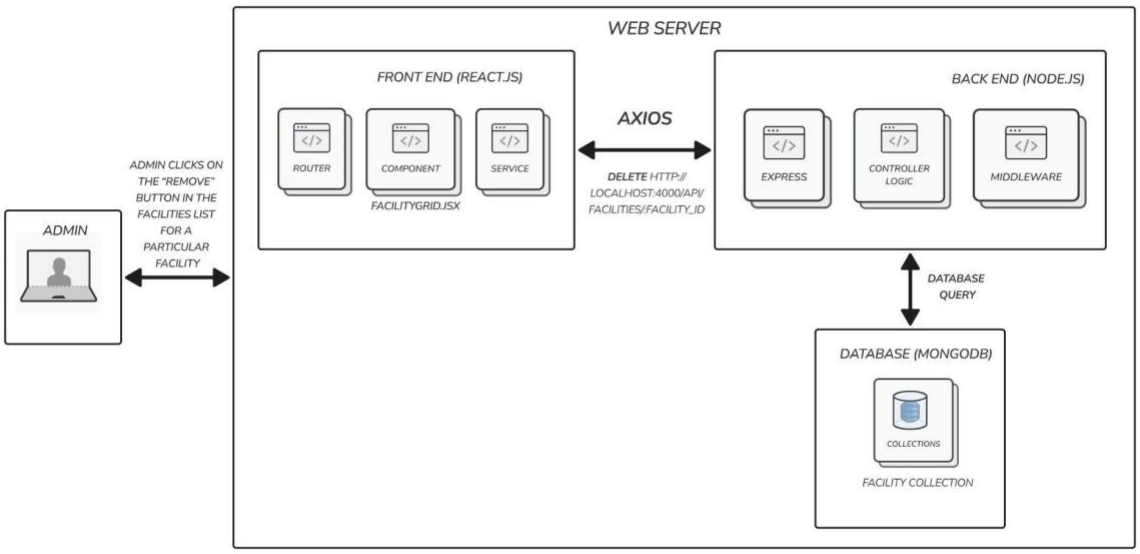

Figure 27: *Removing a facility from the facilities collection.*

In the "Remove Facility" functionality, an administrator can delete an existing facility from the system via the React-based frontend interface. The admin accesses a list of facilities and clicks the "Remove" button next to the facility they wish to delete. This action initiates a DELETE request through Axios to a backend endpoint like /api/facilities/{facility_id}, including the unique ID of the facility to be removed.

The Node.js backend processes this request through controller logic. It then queries the MongoDB Facility collection to locate and soft delete the specified facility document. Upon successful deletion, the backend sends a confirmation response to the frontend, which refreshes the facility list to reflect the removal.

Editing a facility information

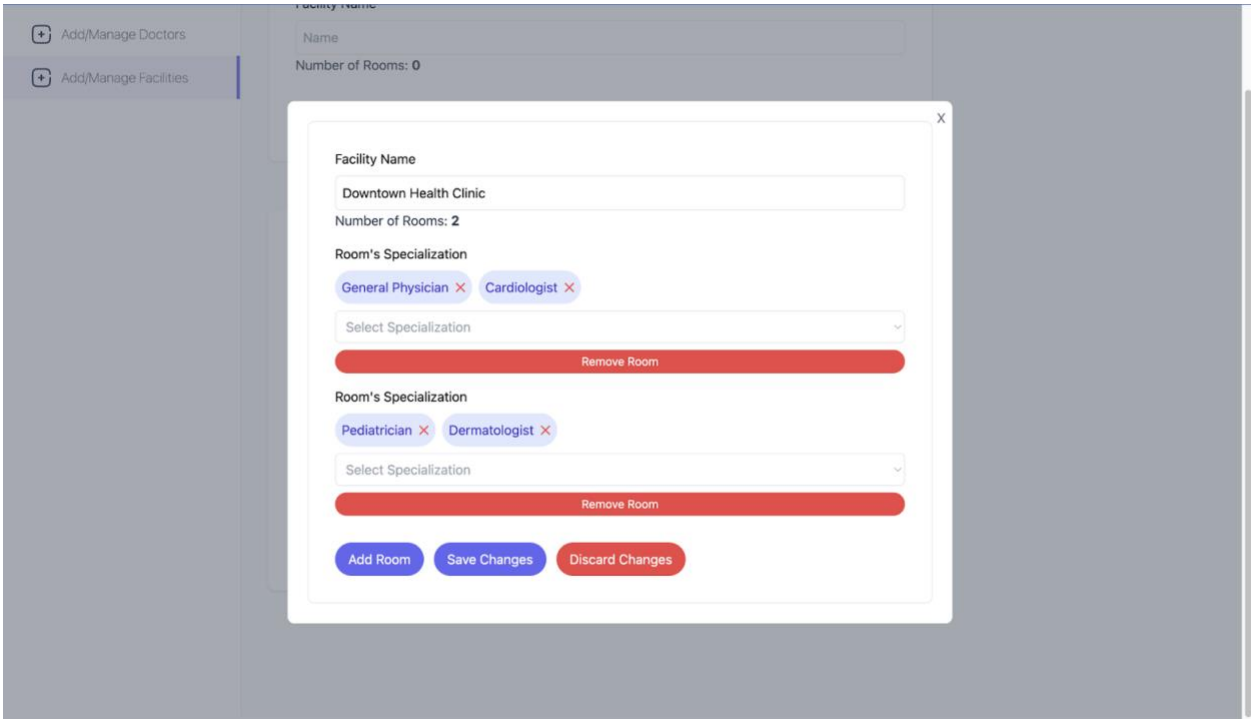

Figure 28: *The pop-up window that appears after clicking the "Edit" button for a facility in the admin interface.*

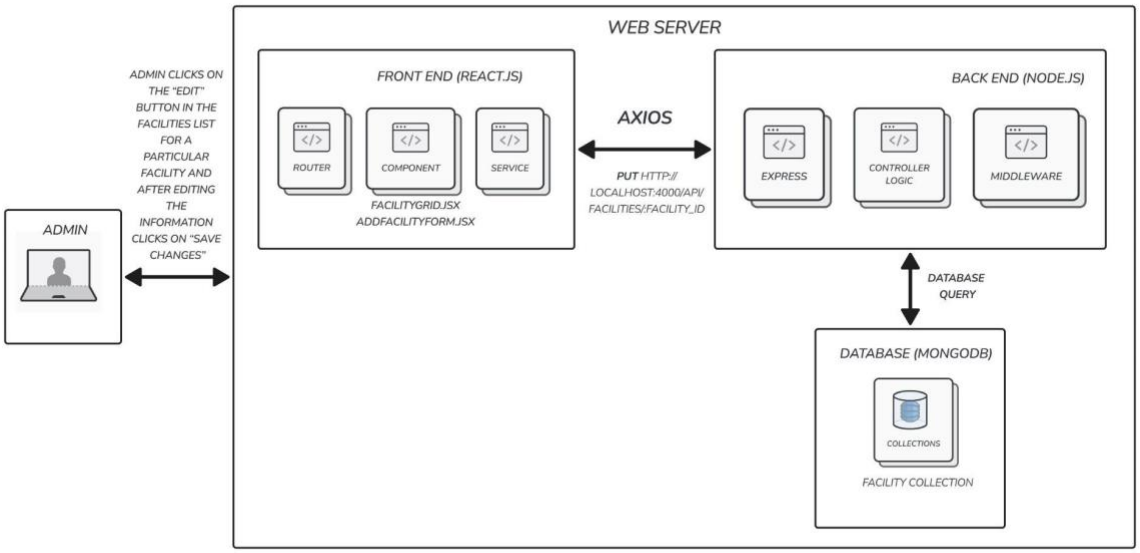

Figure 29: *Editing a facility information from the facilities collection.*

In the "Edit Facility Information" functionality, an administrator can update the details of an existing facility through the React frontend. The admin selects a facility from a list and clicks the "Edit" button, which opens a form pre-filled with the current information of the facility, such as the facility name and room details. The admin can modify this information as needed and click "Save Changes" to submit the updates.

Upon submission, a PUT request is sent via Axios to a backend endpoint like /api/facilities/{facility_id}, including the updated facility data and the facility's unique ID. The Node.js backend processes this request through controller logic, with middleware handling authentication and validation to ensure only authorized edits. The backend then updates the MongoDB Facility collection with the new details. Once successful, a confirmation response is sent back to the frontend, which updates the facility list to reflect the changes. This enables the admin to maintain accurate and up-to-date facility records securely.

Doctor mode:

Doctor Login

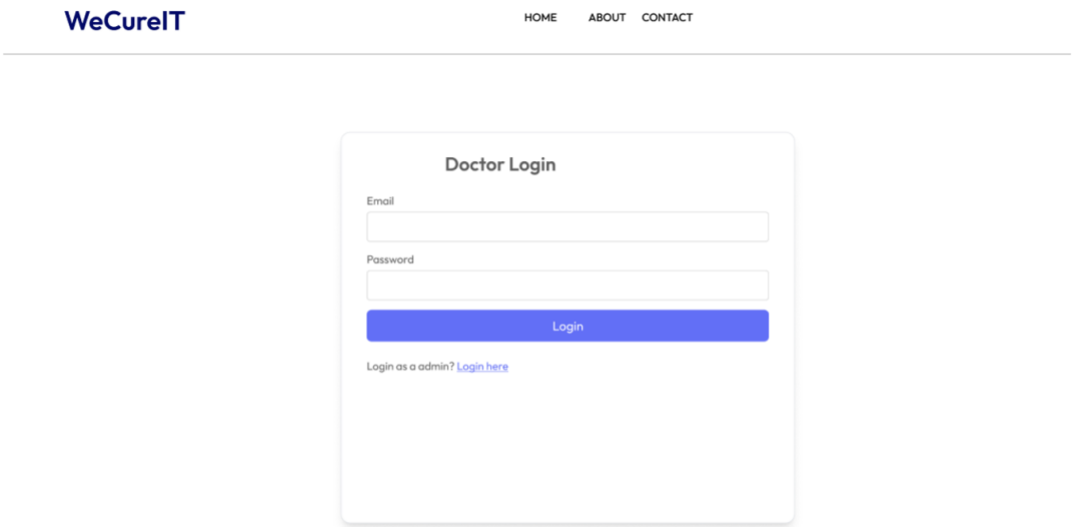

Figure 30: *Doctor Login page.*

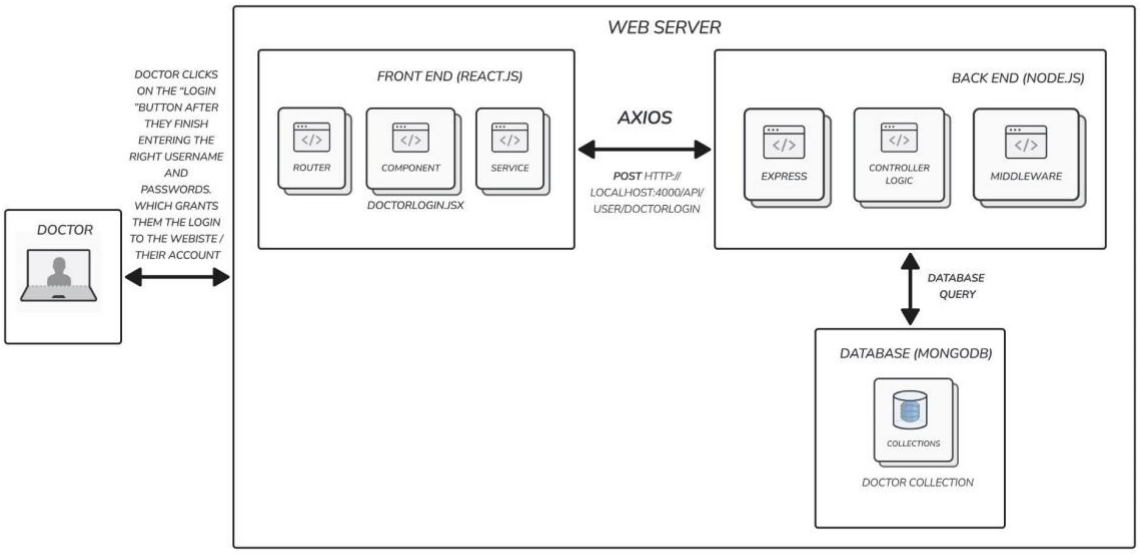

Figure 31: Doctor login

In the "Doctor Login" functionality, doctors can securely log into the system by entering their username and password on the dedicated login page within the React frontend. After clicking the "Login" button, the entered credentials are sent in a POST request via Axios to the backend endpoint, /api/doctor/login.

The Node.js backend receives the request and processes it with controller logic and middleware to validate the credentials and authorize access. It then queries the Doctor collection in MongoDB to verify the provided credentials. If the login is successful, the backend responds with confirmation (such as a token or session data) to grant the doctor access to their account. In case of failed authentication, an error message is displayed to the doctor. This secure login ensures only authorized doctors can access the system’s features intended for their role.

Complete appointment

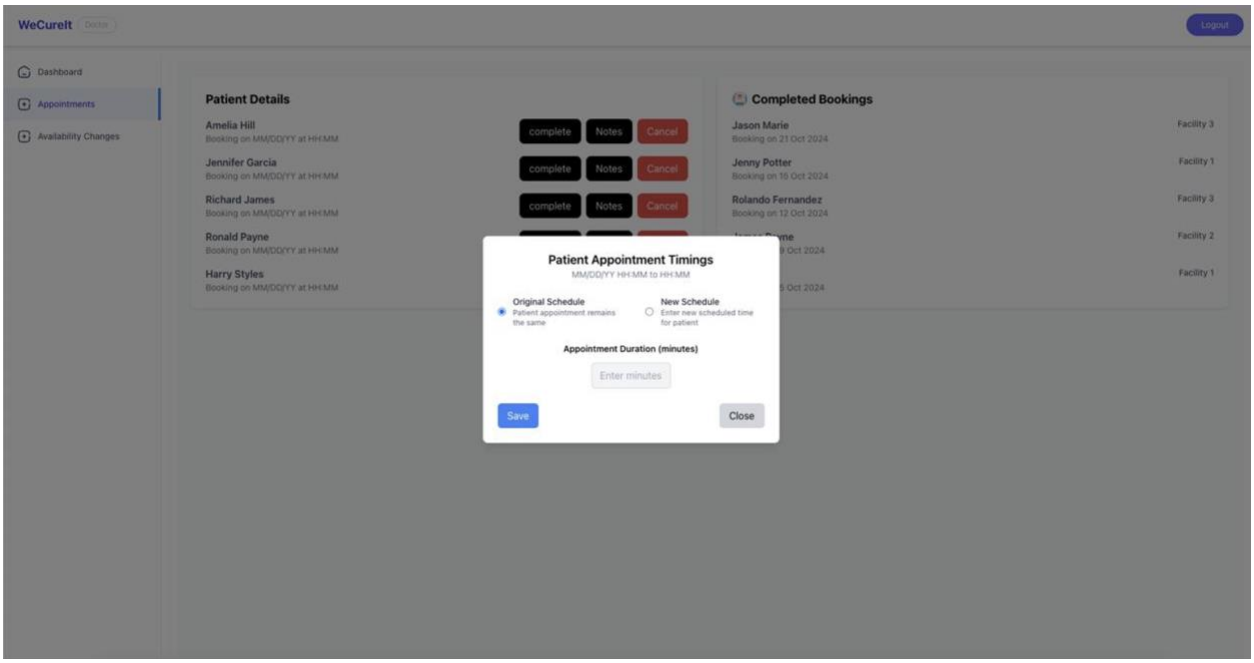

Figure 32: *Patient Appointment Timings Pop-up after clicking 'Complete'.*

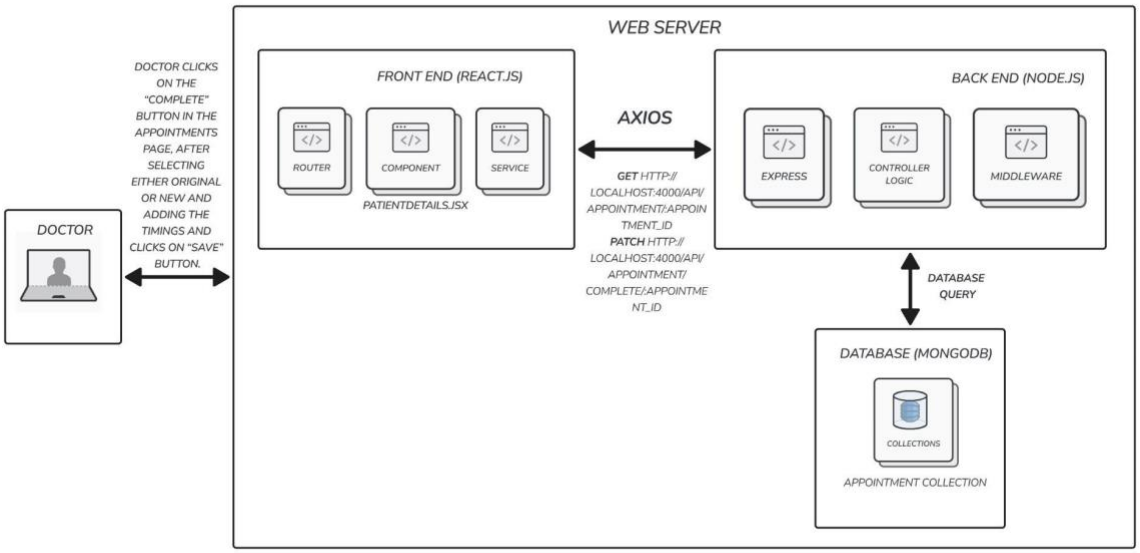

Figure 33: *Marking an appointment complete in the appointment's collection.*

In the "Complete Appointment" functionality within doctor mode, a doctor can mark an appointment as completed through the React frontend. The doctor selects an appointment on the Appointments page, clicks the "Complete" button, and is prompted to review or adjust appointment length timings. After making any updates, the doctor clicks "Save," which triggers an Axios request to update the appointment status.

The request is processed by the Node.js backend. The backend then updates the MongoDB Appointment collection, setting the appointment as complete. Upon a successful update, the backend sends a confirmation response to the frontend, which refreshes the interface to reflect the completion. This allows doctors to efficiently manage appointment statuses and maintain accurate records.

Add/View patient appointment notes

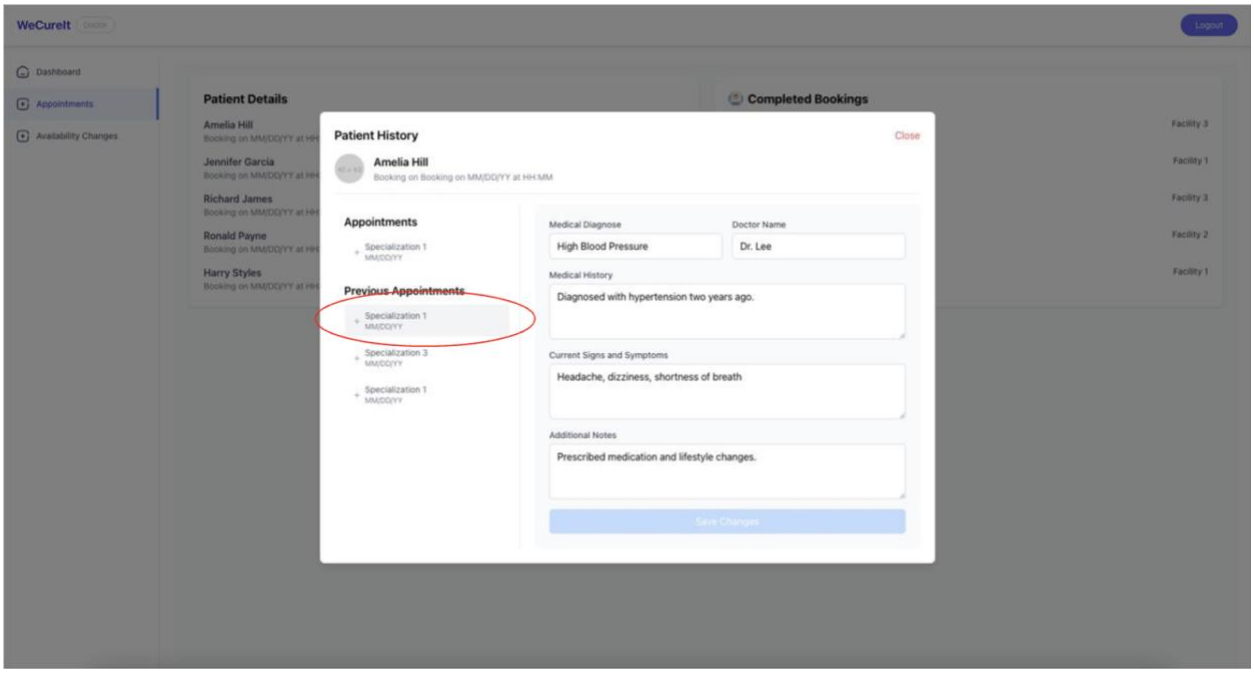

Figure 34 : *Viewing a Previous Appointment details in the Patient Details Notes Page after clicking on "Notes" button.*

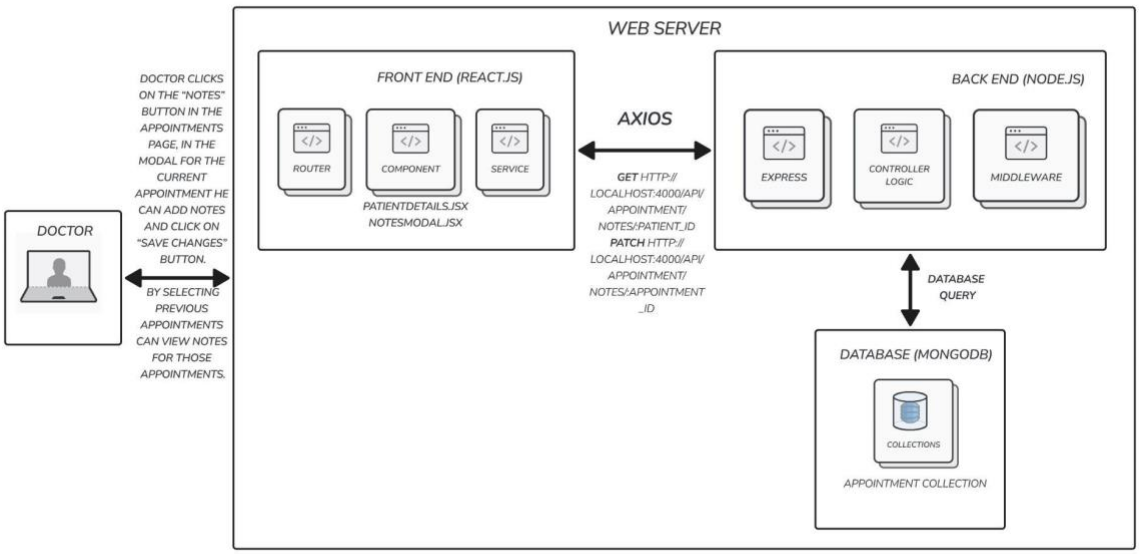

Figure 35: *Adding notes and viewing previous appointments notes to an appointment in the appointment's collection.*

In the "Add/View Patient Appointment Notes" functionality, a doctor can manage notes associated with a patient's appointments through the React frontend. By clicking the "Notes" button on the Appointments page, the doctor opens a modal for the selected appointment, where they can add notes and click "Save Changes" to confirm. Additionally, the doctor can view notes from previous appointments for reference.

This process involves Axios requests to the backend, including a GET request to retrieve existing notes for all previous appointments at endpoint /api/appointment/notes/{patient_id} and a PATCH request to update or add new notes at endpoint /api/appointment/notes/{appointment_id}. The Node.js backend processes these requests, and updates the Appointment collection in MongoDB. Upon a successful update, the frontend refreshes to reflect the newly saved notes, providing a seamless experience for doctors managing patient information.

Cancel Appointment

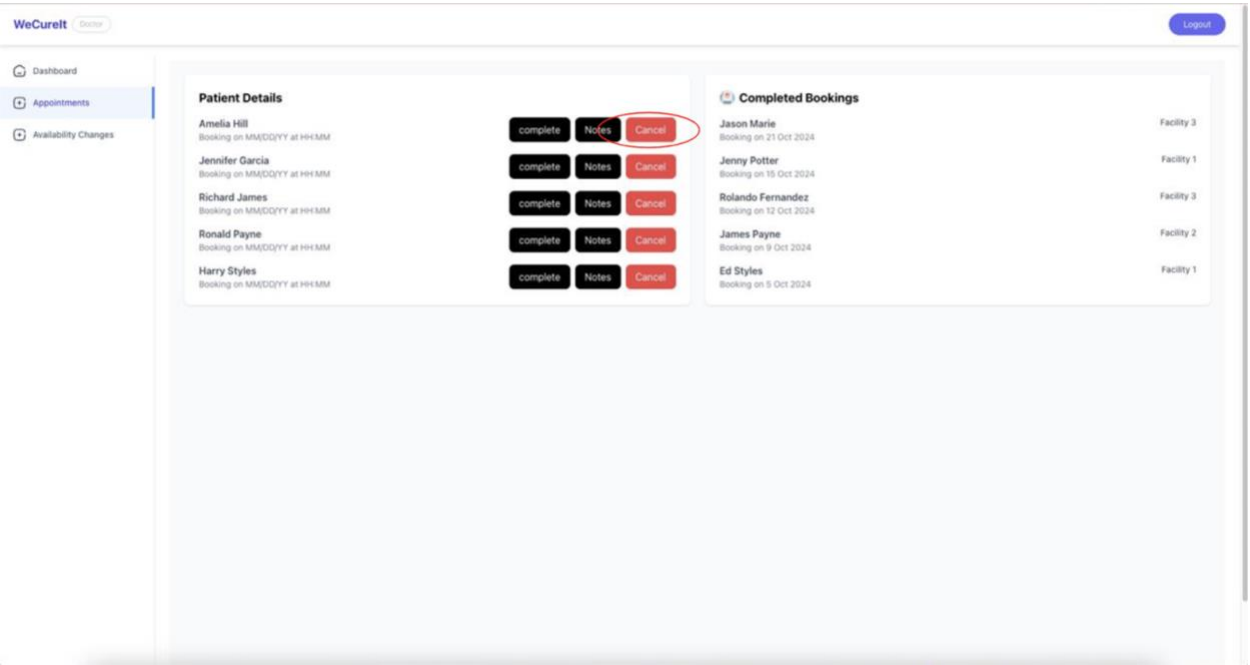

Figure 36 : *Cancel Appointment Option on the Appointments Page*

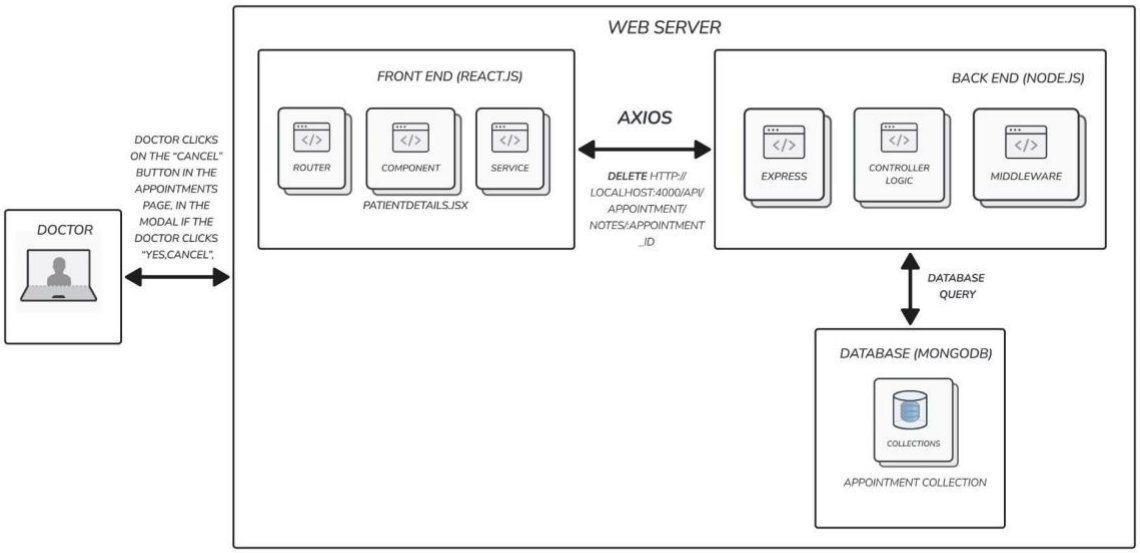

Figure 37: *Marking an appointment canceled in the appointment's collection.*

In the "Cancel Appointment" functionality, a doctor can cancel an upcoming patient appointment through the React frontend. On the Appointments page, the doctor selects an appointment and clicks the "Cancel" button. A confirmation modal appears, and upon confirmation, the doctor finalizes the cancellation.

This action triggers an Axios DELETE request to the backend endpoint, such as /api/appointment/{appointment_id}, where the Node.js backend processes the request through controller logic with middleware for validation and authorization. The backend then soft deletes the specified appointment from the MongoDB Appointment collection. Upon successful deletion, the backend sends a response to the frontend, updating the appointment list to reflect the cancellation and also sends a notification to the user notifying the appointment has been canceled.

Availability form add schedule

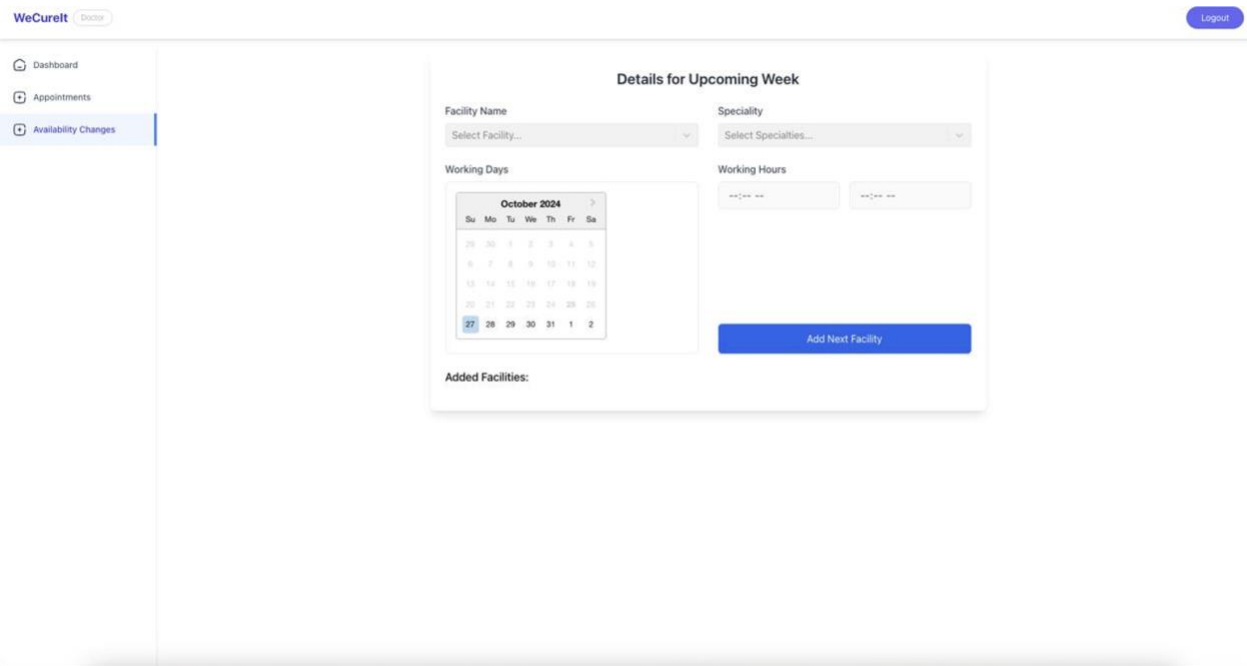

***Figure 38**: Availability Changes Page*

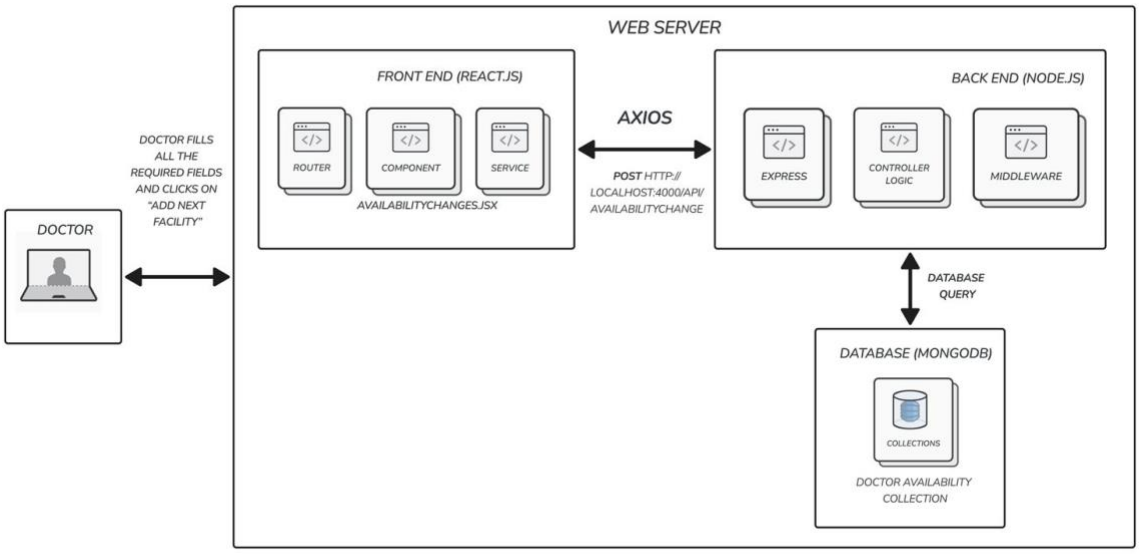

***Figure 39:** Adding a new schedule for a facility in the doctorsAvailability collection.*

In the "Availability Form - Add Schedule" functionality, a doctor can set their availability for specific facilities through the React frontend. The doctor fills out the required fields, such as facility, dates, and hours, and then clicks "Add Next Facility" to submit the schedule. This triggers a POST request via Axios to the backend endpoint, /api/availability/change, with the availability details.

The Node.js backend processes this request using controller logic. The backend then adds the new schedule information to the MongoDB Doctor Availability collection. Upon a successful update, a confirmation is sent back to the frontend, where the interface reflects the newly added schedule.

Availability form edit schedule

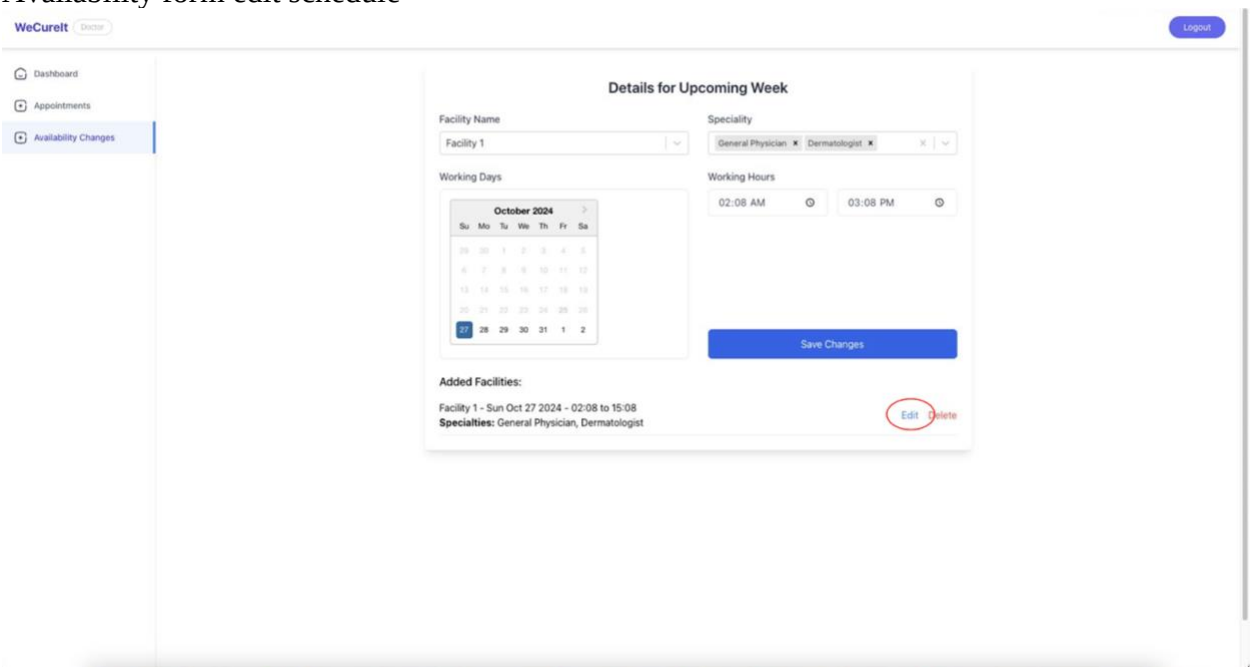

Figure 40*: Editing an Existing Schedule*.

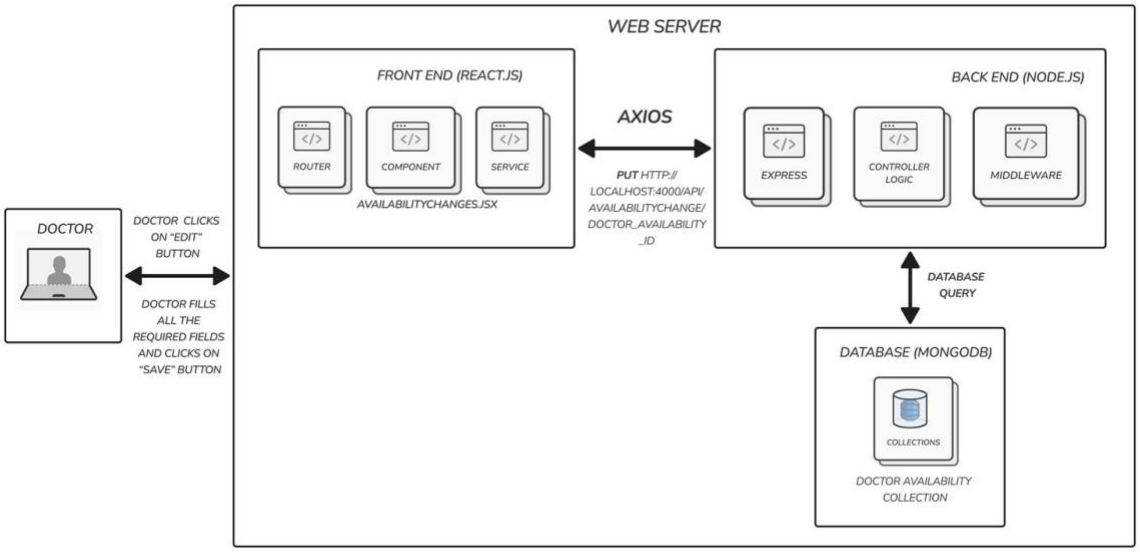

Figure 41: *Editing a schedule for a facility in the doctorsAvailability collection.*

In the "Availability Form - Edit Schedule" functionality, a doctor can update an existing availability schedule through the React frontend. The doctor clicks the "Edit" button next to a scheduled availability entry, which populates the availability form pre-filled with the current schedule details. After making changes, the doctor clicks "Save" to submit the updates.

This action sends a PUT request via Axios to the backend endpoint, such as /api/availability/change/{doctor_availability_id}, with the modified schedule data. The Node.js backend processes the request using controller logic. It then updates the MongoDB Doctor Availability collection with the new information. Upon a successful update, a confirmation response is sent back to the frontend, which refreshes to show the updated schedule, allowing doctors to manage their availability effectively across facilities.

Availability form delete schedule

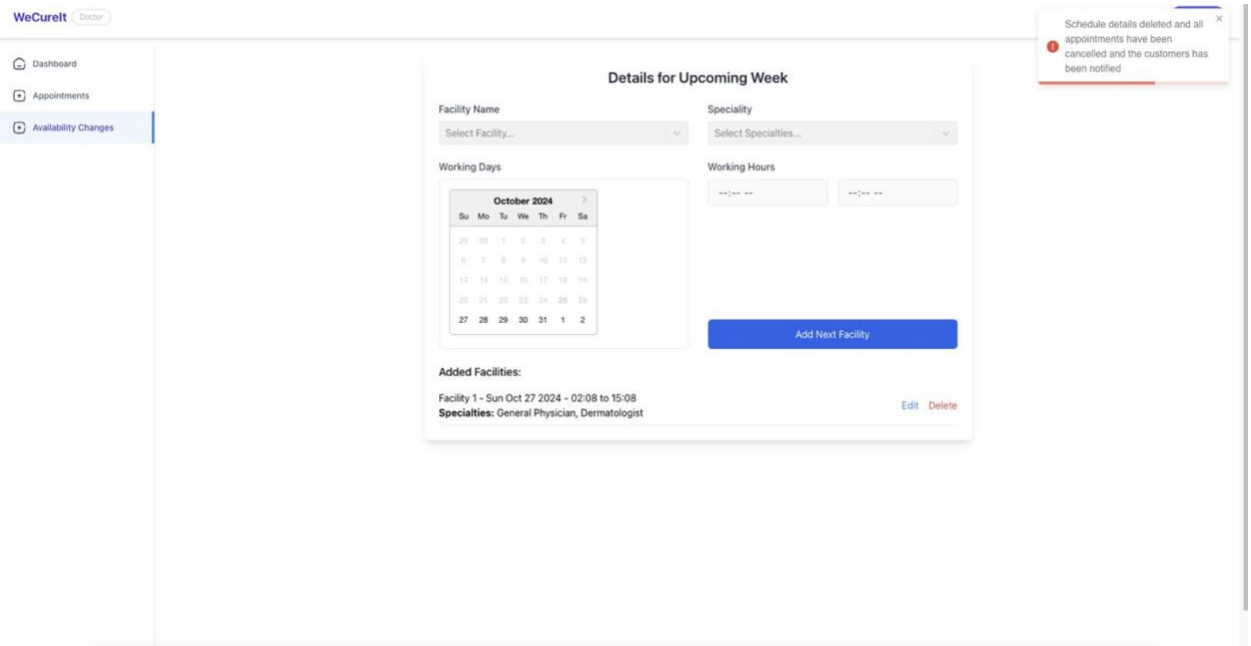

Figure 42: *Removing a specific schedule entry for a facility in the doctorsAvailability collection.*

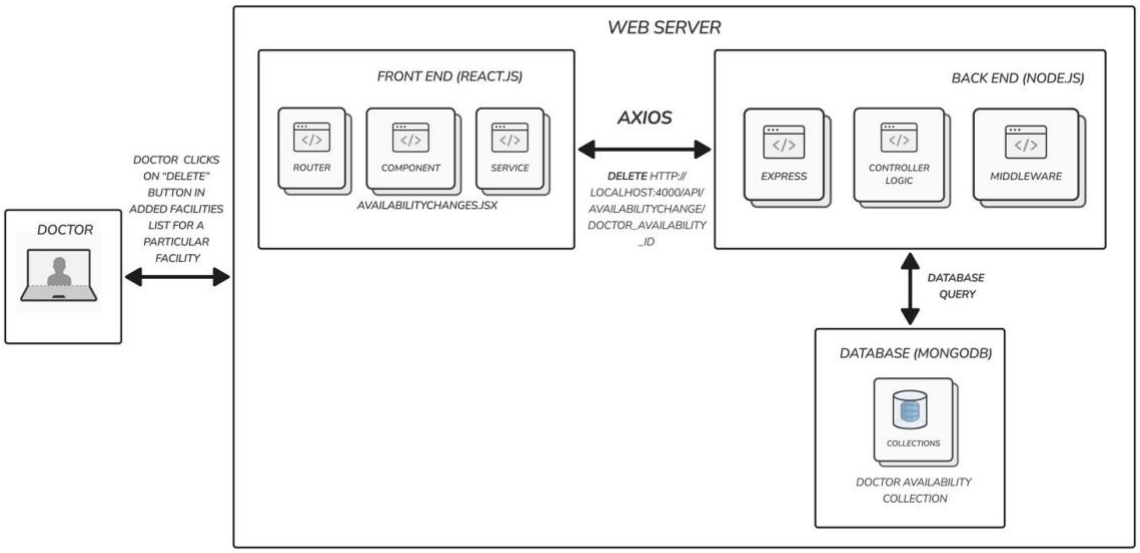

Figure 43: *Removing a schedule for a facility in the doctorsAvailability collection.*

In the "Availability Form - Delete Schedule" functionality, a doctor can remove a specific availability schedule entry through the React frontend. The doctor navigates to their list of scheduled availability and clicks the "Delete" button next to the entry they wish to remove. Upon confirmation, this action initiates a DELETE request via Axios to the backend endpoint, such as /api/availability/change/{doctor_availability_id}, including the unique ID of the availability entry.

The Node.js backend processes this request, with controller logic. The backend then soft deletes the specified entry from the Doctor Availability collection in MongoDB. After successful deletion, the backend sends a confirmation response to the frontend, which updates the list of schedules to reflect the removal.

Availability form copy last week schedule

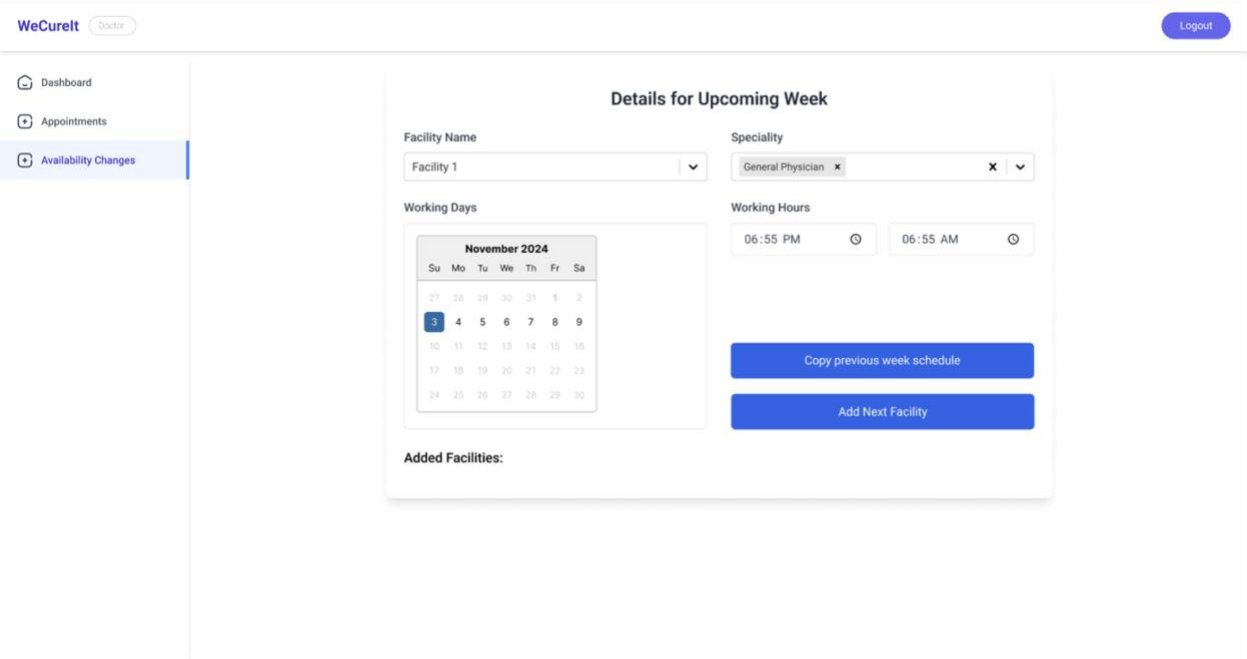

Figure 44: *Availability changes page with the option to "Copy previous week schedule".*

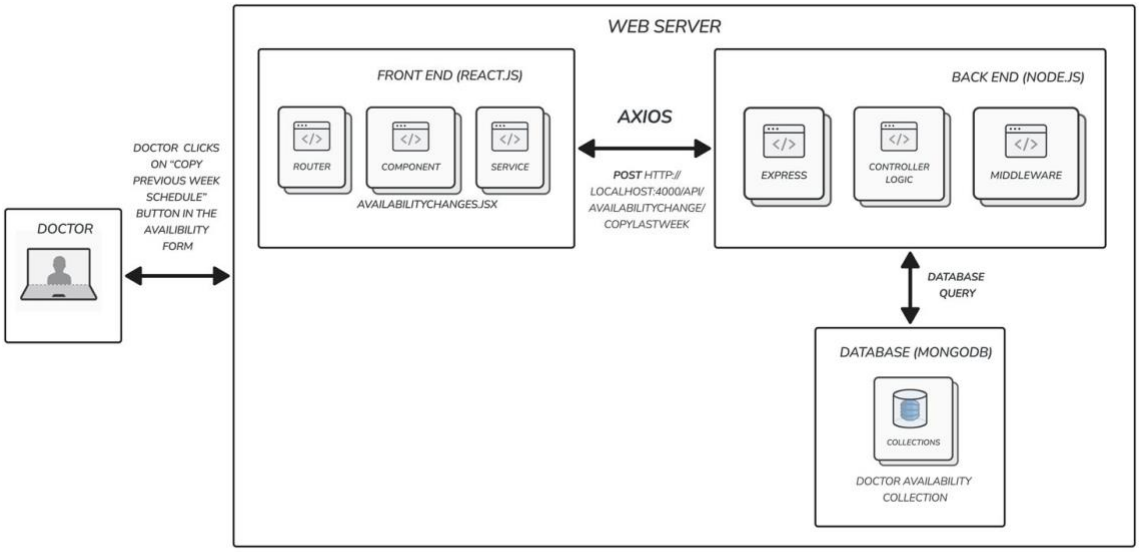

Figure 45: *Copying last week's schedule for a doctor to the upcoming week in the doctorsAvailability collection.*

In the "Availability Form - Copy Last Week Schedule" functionality, a doctor can duplicate their availability from the previous week through the React frontend. By clicking the "Copy Previous Week Schedule" button, the system automatically copies the previous week's availability settings into the current schedule.

This triggers a POST request via Axios to the backend endpoint, such as /api/availability/change/copylastweek, with the doctor's ID or relevant identifier. The Node.js backend processes this request using controller logic. The backend retrieves the doctor's previous week's availability from the MongoDB Doctor Availability collection and duplicates the entries for the current week. Upon successful copying, the backend sends a confirmation to the frontend, which updates to show the copied schedule.